finham park school computing department

OCR A LEVEL COMPUTING
UNIT 3 NEA

PROGRAMMING PROJECT: THE COMPLETE STUDENT GUIDE

UPDATED for 2025

analysis

design

development

evaluation

Produced by J Bridgeman © 2025

© MrBComputingStuff 2022 - 2025

Contents

How to use this guide	page 3
Analysis	pages 4 - 17
Design	pages 18 - 32
Development	pages 33 - 41
Evaluation	pages 42 – 55
Checklist	pages 56 - 58

© MrBComputingStuff 2022 - 2025

How to use this guide

The boxes with numbered prompts are the spec points – this are the descriptors from the A Level specification.

These tell you exactly what OCR expect you to produce in each section of the project – you could use the spec points as headings when planning your written work

The coloured **"What you should do in this section"** boxes explain how to provide the evidence for that particular spec point – specifically, what your work should include.

3. [redacted]

What you should do in this section:

- [redacted]

The lighter coloured boxes/tables give examples of the kind of content you should include in your work or points you should consider writing about. Please note – **these examples are not MODEL ANSWERS** nor do they give an indication of detail

[redacted]	[redacted]
[redacted]	[redacted]
[redacted]	[redacted]

[redacted]

Before you give your completed written work to your teacher to check, try marking it against the assessment criteria. Even though you may find it hard to come up with an accurate grade you should be able to tell if your work:

- Includes all the content required.
- Contains sufficient detail.

© MrBComputingStuff 2022 - 2025

OCR A Level Computing
Unit 3 NEA

STUDENT GUIDE
PART 1

analysis

design

development

evaluation

AO 2.2 Analysis (maximum 10 marks)			
1–2 marks	**3–5 marks**	**6–8 marks**	**9–10 marks**
The candidate will have:			
• Identified some features that make the problem solvable by computational methods. • Identified suitable stakeholders for the project and described them and some of their requirements. • Identified some appropriate features to incorporate into their solution. • Identified some features of the proposed computational solution. • Identified some limitations of the proposed solution. • Identified some requirements for the solution. • Identified some success criteria for the proposed solution.	• Described the features that make the problem solvable by computational methods. • Identified suitable stakeholders for the project and described how they will make use of the proposed solution. • Researched the problem looking at existing solutions to similar problems identifying some appropriate features to incorporate into their solution. • Identified the essential features of the proposed computational solution. • Identified and described some limitations of the proposed solution. • Identified most requirements for the solution. • Identified some measurable success criteria for the proposed solution.	• Described the features that make the problem solvable by computational methods and why it is amenable to a computational approach. • Identified suitable stakeholders for the project and described them and how they will make use of the proposed solution and why it is appropriate to their needs. • Researched the problem in depth looking at existing solutions to similar problems identifying and describing suitable approaches based on this research. • Identified and described the essential features of the proposed computational solution. • Identified and explained any limitations of the proposed solution. • Specified the requirements for the solution including (as appropriate) any hardware and software requirements. • Identified measurable success criteria for the proposed solution.	• Described and justified the features that make the problem solvable by computational methods, explaining why it is amenable to a computational approach. • Identified suitable stakeholders for the project and described them explaining how they will make use of the proposed solution and why it is appropriate to their needs. • Researched the problem in depth looking at existing solutions to similar problems, identifying and justifying suitable approaches based on this research. • Identified the essential features of the proposed computational solution explaining these choices. • Identified and explained with justification any limitations of the proposed solution. • Specified and justified the requirements for the solution including (as appropriate) any hardware and software requirements. • Identified and justified measurable success criteria for the proposed solution.

© MrBComputingStuff 2022 - 2025

3.1.1 PROBLEM IDENTIFICATION

> *"Computer Science is no more about computers than astronomy is about telescopes."*
>
> Edsger W. Dijkstra

Essentially Dijkstra is telling us that if you are thinking about this project from the point of what program you are going to write and how you are going to write it, **then you are doing it all wrong.** Computer Science is about using computers to solve human problems and that should be your starting point.

Real problems always make better projects - this doesn't mean that you can't achieve top marks for the project without a real problem - but an authentic problem should lead to a detailed and thoughtful analysis - and your project will stand or fall on this.

It is ESSENTIAL that you provide a clear definition of the problem. You must assume that someone else reading your analysis (for example a client, your teacher) *does not know anything* about the problem that you are trying to solve.

In the real world, a systems analyst would spend a considerable amount of time looking at whether a problem even suits (or requires) a computerised solution. After all, developing a solution could take a lot of time and money and organisations cannot afford to waste either.

3.1.1 a)
Describe and justify the features that make the problem solvable by computational methods

What you should do in this section:

- **DESCRIBE** the problem that you have chosen to solve - give some context and background.
- Write about the key features of your project that would suit a computational solution and JUSTIFY your reasoning

Here is an example of how a student might approach this, with an extract from a project analysis for a teacher's markbook

```
My programming project will be a teacher markbook program. I have chosen this project
because, having spoken to number of teachers (my A Level subject teachers and my mother,
who is a Primary School teacher), it seems that teachers face a number of challenges and
problems when keeping marks, grades and records of their students.  It also seems that
this seems to take up a significant amount of their time, which could be spent on other
more important tasks such as planning lessons.  Not only do teachers need a way to
record and store student data accurately and efficiently, with the minimum of error, but
they also need to able to analyse this data to assess pupil performance.  Often the
recording of data and the analysis are done separately; a teacher will record a series
of grades/marks in their paper markbook, before carrying out an analysis using a
spreadsheet.  This means that they are recording the same data twice, in two different
places.  It also means that the teacher needs to have sufficient understanding of
spreadsheets in order to perform this analysis correctly. It is of course possible to do
this process manually, but this would both take longer and be more prone to error.
```

© MrBComputingStuff 2022 - 2025

3.1.1 b)
Explain why the problem is amenable to a computational approach.

What you should do in this section:

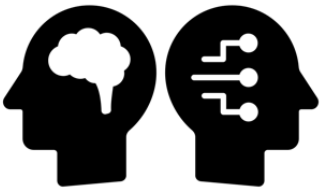
- **EXPLAIN** the features of your problem that lend themselves to a computer solution **AS OPPOSED TO** a manual solution.

You may well have studied the **Turing Machine** or looked at **Hilbert's Decision Problem** in your Computer Science lessons when you have been thinking about the idea of whether all problems can be solved by computers. While this is a very interesting question, let's assume that your potential problem CAN be solved with a computerised solution. (It is extremely unlikely that your teacher would have okayed your idea if this was the case?) The question however, is more about whether it SHOULD be.

- **What are the particular features of your problem that suggest a computational solution?**
- **What manual processes are taking place in the current incarnation of this system, that would be better replaced by computational ones?**

This diagram gives some idea of the kind of tasks that might be best suited to humans / computers respectively. It is not meant to be a definitive list but should give you some ideas.

Tasks suited to humans
'Unstructured' problem solving
Understanding human emotions/empathy/applying common sense
Non-routine physical work
Showing creativity



Tasks suited to computers
Efficient analysis /interpretation of statistics
Performing calculations quickly
Searching/sorting data
Performing repetitive tasks

It is more than likely that your chosen project will be a perfect choice for a computational solution. However, it is possible that a few aspects of your project might lend themselves to a manual solution as opposed to a computational one. If this is the case, you should address this honestly in your write up since you may able to write about them later in your analysis when you are considering the limitations of your solution.

*Whatever your chosen problem, **you must not assume** that the examiner would understand why you feel that it suits a computational solution. You must show this **explicitly** in your writing.*

Here is an example of how a student might approach this, when considering the project about a teacher's markbook.

© MrBComputingStuff 2022 - 2025

Feature	Manual	Computer based	Justification
Facilitating data entry		✔	Data entry could be made simpler with a computer since I could build in validation to reduce possible errors. Manual data entry always carries with it the risk that information could entered incorrectly and reducing human error would be difficult
Storing user data		✔	A computer is ideally suited to data storage, since they cannot misplace or lose it, and can save/retrieve even large amounts of data almost instantly.
Calculating statistics		✔	While it is possible for teachers to work out the performance statistics for their students, this would be so much faster when done with a computer. In addition, if marks or other data were changed, a computer would be able to recalculate results on the fly - this would not be possible with a manual solution
Predicting grade	✔		While I could try and produce an algorithm to produce grade predictions, this would be very challenging to do with any degree of accuracy. A teacher would be able to apply “human” factors, such as their knowledge of the student, as well as their experience of making such predictions, which would likely make a manual solution more accurate.

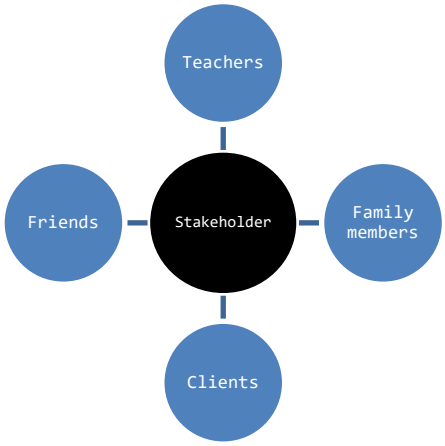
© MrBComputingStuff 2022 - 2025

3.1.2 STAKEHOLDER

> *"If you think your users are idiots, only idiots will use it."*
>
> Linus Toralds

You must identify a stakeholder (or stakeholders) who would be the end user/s of your program. You will need to ensure that your system takes their needs into account. This is not always straightforward - sometimes an end user may not be entirely sure what they want the program to do, or they might have unrealistic expectations of what could be produced.

There are range of possibilities to consider when choosing a stakeholder or stakeholders – but there are also some factors you should bear mind for each one….



AVAILABILITY	Will you have enough access to your stakeholder throughout the project? Will they be available when you need them to be there for interviewing/usability testing?
OBJECTIVITY	Will they give you honest, unbiased opinions about your work?
KNOWLEDGE	Do they have realistic/sensible expectations of what the project should involve? Can they answer technical questions?

3.1.2 a)
Identify and describe who will have an interest in the solution, and how it will meet their needs.

What you should do in this section:

- Write about **who** will use the system that you are creating, and
- Write about **how** they will use it (i.e. **usability** – including any **accessibility issues**)

If you have a sound idea for your project, with a well-reasoned use case and a clear stakeholder in mind, you should be able to write about exactly how the system will be used and by whom. However, it is very likely that you will have multiple stakeholders in mind, in which case you will need to bear in mind the fact that **different stakeholders might have different requirements** from your system…

© MrBComputingStuff 2022 - 2025

The example below shows how different groups of users may use an operating system.

Power User	Managing and configuring hardware and software Network security Providing multi-user support
Home user	Opening programs Managing files Accessing the internet

You should also determine whether any of the prospective users have **accessibility requirements** that would affect their ability to use the program. Accessibility requirements could include factors such as:

Physical interaction	Are there options for control/data entry? (i.e. for left/right handed users, or younger/older users who may struggle to use a mouse) How can you simplify the process of entering data into your system? What can be done to minimise the chance of making data entry mistakes?
Presentation of text	Are there options for users to change size/colour of text? Is there an appropriate contrast between foreground/background colours? Can this be adjusted?
Multimedia	Can playback of audio/video be paused? Can volume be adjusted or muted altogether?
User knowledge /experience	Does the design of program take into account the age/knowledge of the person using it? What help is available to the user from within the program?

Your DESIGN will need to include any appropriate accessibility features. You will need to ensure that you TEST and EVALUTE the usability of your program later.

Collecting information from your stakeholders

Your stakeholders can provide very useful insights regarding a range of important information – for example, what the system should contain, what accessibility needs must be accounted for. After you have identified your stakeholders, you should think carefully about all the information that you will need to collect from them and then work out how/when you will do so.

The two easiest methods of data collection are:

- **Interview**
- **Questionnaires**

You could use either method or combination of both. However, whichever method or methods are chosen, there are things to consider:

© MrBComputingStuff 2022 - 2025

INTERVIEW	QUESTIONNAIRE
Will you interview your stakeholders together or separately? Have you arranged an appropriate time/place for the interview to take place? Have you planned what questions you will ask? Have you considered how you will collate and write about the findings of your interview?	Will you use paper questionnaires or online? (i.e. Google Forms) Have you used the right mix of open and closed questions? Have you allowed sufficient time for your stakeholder to respond to your questionnaire? Have you allowed enough time for you to collate the results?

Good question design is also important, for both interviews and questionnaires. You must ensure that you ask for the information that you need and not for the information you don't.

You must also consider how easy your questions are to answer, given the age and experience of the users*. For example, you should avoid using technical terms that the user might not understand.*

- Make sure that the findings from your stakeholder research appear in your work.
- It is important that it is clear about how you have made any decisions or come to any conclusions in your analysis – refer to your questionnaire/interview data as appropriate.
- You can include a copy of your questionnaire as an appendix when you have completed your project.

© MrBComputingStuff 2022 - 2025

Below are some example questions and question styles – they should give you an idea of what you should do (as well as what you shouldn't)

EXAMPLE QUESTION	GOOD OR BAD?
GENDER ☐Male ☐Female ☐Other	**POOR QUESTION CHOICE** *Is this question necessary? Is it just being included because it is often used on questionnaires?* *Is the user's gender a relevant factor in solving a problem?*
What makes a good UI?	**POOR QUESTION DESIGN** *This question is far too open. As an open question it would also be time-consuming to answer and more challenging to analyse.* *In addition, the user may not even have a clear idea of what a UI is, much less what a good UI is. In its present form it is a very abstract question.*
Rate the following factors on a scale of 1- 10 (1 being VERY IMPORTANT, 10 being NOT AT ALL IMPORTANT) Options for text size are included ☐ On-screen choices are listed alphabetically ☐ Menu choices can be selected via keyboard shortcut or mouse ☐ Each menu screen has a "dark mode" option which can be toggled on/off ☐	**GOOD QUESTION DESIGN** *This is a better version of the question above.* *The user has been given clear choices and can use the "ratings" system to express their degree of preference.* *If you use a numbered system, always use an even number like 10 – many people will choose the middle option by default if an odd number is used. For example, if asked to rate something on a scale of 1 – 5 people may choose 3 just because it is in the middle!*
Will on screen instructions be required? ☐ YES ☐ NO	**POOR QUESTION DESIGN** While closed questions with YES/NO answers are easy for the stakeholder to answer and straightforward to analyse, they yield very little useful information.
What kind of on-screen instructions would you prefer? An optional pop-up on each screen ☐ A separate tutorial section ☐ "TOP TIPS" that pop up between menu screens ☐ None ☐ OTHER (Please specify) _____	**GOOD QUESTION DESIGN** This is a better version of the question above. A range of choices will result in more useful set of responses from the user. It is important to give another option for multi-choice questions like this

© MrBComputingStuff 2022 - 2025

3.1.3 RESEARCH THE PROBLEM

> *"If we knew what it is we were doing, it would not be called research. Would it?"*
>
> Albert Einstein

Thorough research is a vital part of any project analysis. It would be foolish to begin designing a solution to a problem without first ensuring that you understand every aspect of that problem.

3.1.3 a) Research the problem and solutions to similar problems to identify and justify suitable approaches to a solution

What you should do in this section:

- **Find** and **write about** examples of problems that are similar to yours
- **IDENTIFY AND EXPAIN** features of these solutions that could be incorporated into your project
- **Draw on your research to justify** your reasons for including these features

The example below shows how a student producing a top-down dungeon crawler might undertake research by looking at similar games. The student would then be able to use their research to help them identify:

- **Key features** of each of the solutions (with some screenshot evidence)
- **Ideas that could be incorporated** into the student's project (**with reasons why**)
- **Ideas that could not be incorporated** into the student's project (**again, with justification** of why these ideas would be omitted)

You would be expected to look at these existing solutions in more detail - but the example shown here does give some idea of the approach you could take.

Gauntlet was a 1985 arcade game, produced by Atari Games and later ported to home computers including the Commodore 64. It featured 1 -4 players assuming the role of "fantasy" characters, battling their way through dungeons filled with hordes of monsters.

The levels were not randomly/procedurally generated and I think I may need to consider this approach for my own game, due to the complexities of creating procedurally based levels.

The gameplay is simple - it is entirely combat based - which I could replicate, but supporting multiplayer gameplay may be more of a challenge.

Heroes of Loot 2 is also a dungeon based hack-and-slash, which was produced by OrangePixel in 2016, initially for mobile with later releases for PC and console. There are clear nods to Gauntlet (with the fantasy setting and the hack-and-slash combat) but game includes features that Gauntlet doesn't (for example, collecting treasure/loot, unlocking other characters and some simple puzzle solving and updating the characters' level over time, in a "rogue like" style. I like the idea of including these features.

The main difference is that game supports 1-2 players (not 1-4) and that levels are procedurally generated. This would be harder to implement given that the levels cannot simply be randomised if the game is to be balanced and ensure levels that are not impossible to complete.

Diablo 3 is also a dungeon based hack-and-slash, released by Blizzard games in 2012. The game was released on PC/consoles. Again it supports single and multiplayer modes, but with a much wider range of character classes and much more emphasis on "levelling up" with the character being able to make use of skill trees and a wide range of weapons and abilities. I could consider including these ideas if I make use a more modest range of skills and weapons.

Obviously the main difference is the use of 3D graphics and the game makes use of a sophisticated physics engine and destructible environments. Producing a game in this style would be extremely challenging and even if I was able to do so, I would struggle to complete the game in time.

© MrBComputingStuff 2022 - 2025

3.1.3 b) Describe the essential features of a computational solution, and explain these choices

What you should do in this section:

- Produced **a detailed and focused** piece of writing **EXPLAINING** which features of your project are **essential**
- **EXPLAIN** clearly WHY the features you have included are essential

Now that you have you a clear idea of what your project will be about and who it is aimed at, you should be able to break it down into the key features it will need to include. Your work in this section will need to include:

- **All the features that you intend to include in your solution**
- **What that feature will do**
- **Why the feature is needed** [don't forget about the needs of your users!]

Remember that this is an analysis NOT a design – you should not be considering HOW you will produce each of your features (at least not yet)

The example here shows how a student identifies, describes, and justifies some of the features that they have decided to include in their solution.

Feature	Description	Justification
Save game feature	This would allow players to save their progress in the game -for example, their current score, level and character stats. Progress would be saved to a file which would then be loaded when the player returns to the game	Without a save game option the player would lose all their progress which would not only be frustrating and would impact on their enjoyment, but would also mean players would have to play each game until its completion, making longer gameplay sessions which is bad for player wellbeing.
Control options	This would allow users to choose either mouse or keyboard control. Users who select keyboard control and re-define the keys if they want to.	Being able to choose a control scheme is important since it will directly impact the user's ability to play the game. In addition, this would enable me to account for players who are both left and right-handed and who may struggle to use either a mouse or keyboard.

© MrBComputingStuff 2022 - 2025

3.1.3 c) Explain the limitations of the proposed solution.

What you should do in this section:

- **REFLECT** on and **EXPLAIN** the features you have decided NOT to include in your project
- Also **consider** any features that you are including that have some limitations.
- Be able to **EXPLAIN AND GIVE REASONS** for the above.

It is inevitable that your project will have some limitations. During the process of exploring existing solutions and developing your own ideas, you will have to make – possibly difficult – decisions about features you will have to leave out, or features that might have to contain limited functionality.

If your project doesn't have any limitations, there are two possibilities:

1. **You have not been realistic about what you are able to produce.**
2. **Your idea is too simplistic and doesn't have the scope necessary for a proper A Level project.**

It is FINE to identify limitations for your project. You will be able to write about these in the evaluation phase of the project when you are asked to consider possible areas of development.

In the example below, the student has reflected on their original intentions as well as the research they did into similar problems, when considering possible limitations to their project.

LIMITATION	REASONING
Levels will need to be pre-designed not procedurally generated	Having done some research (including watching OrangePixel's vlog posts on this topic) I have decided that it would be too complex to create procedurally generated levels. Procedurally generated levels cannot be completely randomised, since there would be a risk of levels that were impossible to complete. The algorithm required to ensure that every level was balanced in terms of its mix of power-ups, loot, monsters etc, would need a lot of time to get right. I am concerned that my focus would be on this rather than getting the core game mechanics working. To this end, I will leave procedurally generated levels as a future development target
The game will only contain 5 levels in one world – not five levels in five worlds.	This limitation has occurred as result of the previous one. Since my levels will not now be procedurally generated, it would not be realistic to aim to have the same number of levels that this would have allowed. I will need to ensure that the pre-designed levels that I do produce will contain sufficient mechanics and gameplay evidence to produce a satisfactory gameplay experience.

© MrBComputingStuff 2022 - 2025

3.1.4 SPECIFY THE PROPOSED SOLUTION

"I never worry about the problem. I worry about the solution."

Shaquille O'Neal

3.1.4 a) Specify and justify the solution requirements including hardware and software configuration (if appropriate).

What you should do in this section:

- **IDENTIFY and EXPLAIN** any requirements that have been agreed with your STAKEHOLDER/S
- **IDENTIFY and EXPLAIN** the functional requirements of your system
- **IDENTIFY and EXPLAIN** any hardware/software requirements of your system
-

The "solution requirements" can be organised into THREE discrete categories:

- **Stakeholder requirements**: Specific requirements agreed with your stakeholder/s.
- **Functional requirements**: All of the things that your system will need to be able to do.
- **Hardware and software requirements:** The hardware and software that is needed in order for the system to run properly.

You need to think carefully about the hardware and software requirements also need to be aware of any hardware or OS versions that your system would be INCOMPATIBLE with and make sure that you are not making unrealistic or unnecessary demands of the user.

For example, just because you may have developed the program on an M2 MacBook Pro, that alone does not mean that your users will need the same machine in order to run the program!

WHAT WILL THE USER NEED IN ORDER TO RUN THE SOFTWARE?	
HARDWARE..	**SOFTWARE..**
Processor?	Operating system?
RAM?	Application software?
Multimedia? (i.e. graphics/soundcards)	Utility software?
Input devices? (keyboard, mouse, controller etc)	
Output devices? (speakers, monitor)	

© MrBComputingStuff 2022 - 2025

Below is an extract from a table of requirements, created by the student who is making the retro arcade game.

Notice how each of the requirements in the example is categorised and explained in full.

REQUIREMENT	CATEGORY	EXPLANATION
The program must have left/right handed control schemes.	**Stakeholder**	I have a mix of left and right-handed stakeholders and it was important to both groups to have a control scheme that would make the game playable for them
The program must have a menu screen with PLAY GAME/CHOOSE CONTROLS/QUIT options	**Functionality**	The game needs a menu screen in order for users to make it straightforward to play or quit the game. As I need to provide control options, it would seem sensible to make this accessible from the menu screen to keep all the options in the same place.
Python IDE running at least Python 3.0, with PyGame installed	**Hardware/software**	Any Python IDE that supports Python 3.0 and upwards will be fine, as long as the PyGame library can be installed. The user would need to ensure that they have the admin rights to install this on their computer.
Graphics/sound card	**Hardware/software**	The game will feature 2D graphics as well as sound effects. While it will not require powerful hardware to run, it must be able to handle simple animated sprites as well as sound effects/music

3.1.4b) Identify and justify measurable success criteria for the proposed solution.

What you should do in this section:

- Produce a list of MEASURABLE success criteria which can be used to judge how successfully you have implemented your project.
- **JUSTIFY** each success criteria

At this point, you should have a detailed and well thought through analysis of the problem that you intend to solve for your NEA.

Before moving on to the design stage, you need to produce a set of criteria by which the success (or not!) of your project can be judged. You will need to return to these criteria in the final part of the NEA – your evaluation – so that you can make *informed judgements* regarding how successful the outcomes of your project are. Your success criteria need to be thoughtfully designed or you run the risk that it will be difficult (or impossible) to prove definitively that you have met them.

Try and set SMART targets as your success criteria:

S PECIFIC
M EASURABLE
A CHIEVABLE
R EALISTIC
T IMEBOUND

© MrBComputingStuff 2022 - 2025

Here some examples of both good – and bad – success criteria

Criteria		Good example	Bad example
SPECIFIC	Each objective must be **single purpose**, and described clearly and with a level of detail that ensures unambiguity.	*There should be both a single player and two player option*	*Gameplay options must include the possibility of more than one player*
MEASURABLE	Your success criteria should be CLEARLY MEASURABLE with quantifiable outcomes	*The data should be sorted in under 10 seconds*	*The data should be sorted quickly*
ACHIEVABLE	The criteria you set must be ones that can actually be achieved. A number of factors will affect this - for example, your programming ability, the time available and the resources at your disposal	*The game should support on-device multiplayer for up to 4 players*	*The game should be able to support online multiplayer for up to 100 players*
REALISTIC	The system should not place unrealistic demands on end users - you should think about what skills and knowledge, hardware and software your end users are likely to have	*The target system should be a PC capable of running Windows 10*	*The target system should be:* *CPU: Intel Core i7-12700* *RAM: 64GB* *Graphics Card: Nvidia GeForce 3090* *Storage: 2TB SSD*
TIMEBOUND	Any success criteria you set must be achievable before the project deadline that your teacher will give you. Make that the amount of work you give yourself reflects the time you have to complete it!	*The game should contain 3 levels*	*The game should contain 5 "worlds", each world containing 5 levels with a central "hub"*

While you are choosing your SMART criteria, you must consider:

a) How you will be able to provide evidence that you have achieved it
b) How you will justify the inclusion of each criterion

Don't forget the importance of your end user/stakeholder in evidencing whether you have met your criteria. You should also try to consider the different ways of generating evidence to prove that you have met your criteria… can you avoid "death by screenshot"?

Here are possible success criteria that could be set for the dungeon crawler project.

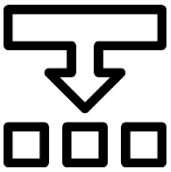
SUCCESS CRITERIA	EVIDENCED BY	JUSTIFICATION
The player can change game controls	Screenshots/stakeholder witness statement	The ability to change the default control scheme has been highlighted as an important requirement by the users and is a key usability factor
Collision detection (between player / environment) works in all levels	Screen recording/stakeholder witness statement	Faulty collision detection would essentially mean a broken, unusable game. It has to work properly on each level

© MrBComputingStuff 2022 - 2025

AO 3.1 Design (maximum 15 marks)			
1–4 marks	**5–8 marks**	**9–12 marks**	**13–15 marks**
The candidate will have:			
• Described elements of the solution using algorithms. • Described some usability features to be included in the solution. • Identified the key variables / data structures / classes (as appropriate to the proposed solution). • Identified some test data to be used during the iterative or post development phase of the process. •	• Broken the problem down systematically into a series of smaller problems suitable for computational solutions describing the process. • Defined the structure of the solution to be developed. • Described the solution fully using appropriate and accurate algorithms. • Described the usability features to be included in the solution. • Identified the key variables / data structures / classes (as appropriate to the proposed solution) and any necessary validation. • Identified the test data to be used during the iterative development of the solution. • Identified any further data to be used in the post development phase.	• Broken the problem down systematically into a series of smaller problems suitable for computational solutions explaining the process. • Defined in detail the structure of the solution to be developed. • Described the solution fully using appropriate and accurate algorithms explaining how these algorithms form a complete solution to the problem. • Described, explaining choices made, the usability features to be included in the solution. • Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) explaining any necessary validation. • Identified and justified the test data to be used during the iterative development of the solution. • Identified and justified any further data to be used in the post development phase.	• Broken the problem down systematically into a series of smaller problems suitable for computational solutions, explaining and justifying the process. • Defined in detail the structure of the solution to be developed. • Described the solution fully using appropriate and accurate algorithms justifying how these algorithms form a complete solution to the problem. • Described, justifying choices made, the usability features to be included in the solution. • Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) justifying and explaining any necessary validation. • Identified and justified the test data to be used during the iterative development of the solution. • Identified and justified any further data to be used in the post development phase.

© MrBComputingStuff 2022 - 2025

3.2.1 DECOMPOSE THE PROBLEM

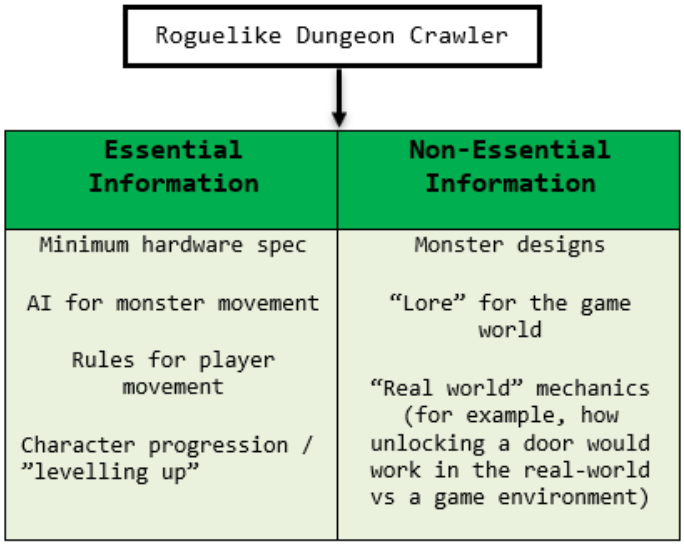


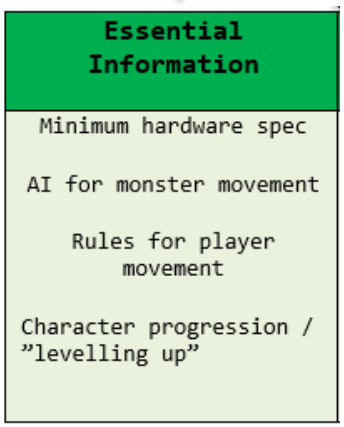
> "Computer Science is a science of abstraction – creating the right model for a program and devising the appropriate mechanizable techniques to solve it"
>
> **Alfred Aho**

Before a line of code is written, before a single algorithm is planned, you must have taken the project idea that you wrote about in such detail for your analysis. and taken it all apart. It is essential to have a clear breakdown of every part of your system and that you are able to communicate this to the examiner.

However, it is harder to decompose a problem when we don't fully understand it. Even though you aren't specifically asked to produce an abstraction for this coursework, going through the process of applying abstraction to your problem will result in you developing a better understanding of what the key parts of your system actually are.

For example, the student designing a roguelike dungeon crawler might find it useful to apply some abstraction, before carrying out a decomposition.





These are the parts of the problem that the programmer can focus on when attempting to decompose the problem

© MrBComputingStuff 2022 - 2025

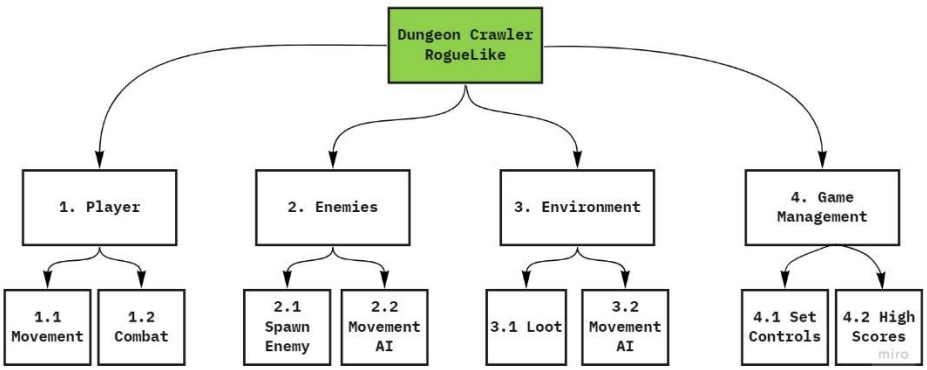
3.2.1 a)
Break down the problem into smaller parts suitable for computational solutions justifying any decisions made.

What you should do in this section:

- Produce a DECOMPOSITION DIAGRAM
- **JUSTIFY** your reasons for including each of the "sub problems" identified in your decomposition.

Remember that a decomposition diagram starts with your "overall" problem and then breaks it down into smaller and smaller problems until it can't be broken down any further.

Below is an example of a decomposition for the rogue-like Dungeon Crawler – yours should be more detailed than this.



Remember:

A decomposition diagram is NOT a flowchart – it depicts **WHAT** your system will do not HOW it will do it.

It is a good idea to a use consistent numbering scheme (as seen in this example) so that you cross reference later work back to this diagram.

It is worth remembering that you should be aiming to work towards a **modular solution.**

Once you have produced your decomposition diagram, you can write about why you have included each of sub-problems you have identified, ensuring that you justify each one.

Below is an extract from an example of what this might look like for the Dungeon Crawler decomposition diagram – note how the numbering is consistent with the diagram.

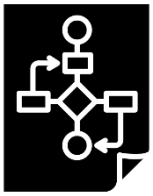
1.1 Movement	I will have to program movement for the player's character because the basic gameplay functions of this type of game would not work without player movement. As it will be "top down" the movement will have to be multi-directional.
1.2 Combat	As I have clearly identified combat the tropes that one would expect to see in a dungeon crawler, I will need to ensure that I include this in my solution. I will need to ensure that I can support both close-combat (melee) and ranged combat (allowing the player to utilise bows or magic. Without combat, my game would really just be a maze/exploration game and I would not be able to meet the success criteria that I identified in my analysis.

Here is a weakness of this particular decomposition – combat could easily have been broken down into two smaller problems (melee combat and ranged combat) allowing for a more focused explanation of each one. As it is, having a single, problem will make it harder to solve.

© MrBComputingStuff 2022 - 2025

3.2.2 DESCRIBE THE SOLUTION

3.2.2 a) Explain and justify the structure of the solution.



```
"Design is a funny word. Some
people think design means how
it looks. But of course, if you
dig deeper, it's really how it
works."

Steve Jobs
```

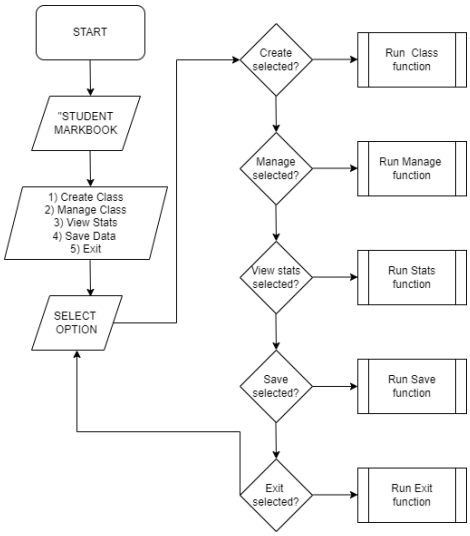
The word “explain” is misleading here – you do not need to produce a detailed, written description of your system. You will need to produce a representation of each part of your system and show how it structured, ensuring the relationship between each part of the system is clearly explained and established.

OCR do not give one, specific method for doing this. How you choose to do this will depend on a number of things and there are a range of methods for representing how your system fits together. These include:

- **System flowcharts**
- **Web-page structure diagrams**
- **Entity Relationship Diagrams/Database tables** (if you are working in SQL)
- **Input/Process/Storage/Output diagrams** (great for systems that involve a lot of data handling)
- **Data Flow Diagrams**

Please note – the emphasis is very much on explaining how the system fits together, not how it works. You can do that through the algorithm design, which comes next. This means that while you CAN use a method such as a flowchart (or even some pseudocode), you should do so purely to enable you to show how each “module” of your program relates to the others.

Here is a flowchart representing the menu screen of the student markbook program, along with an explanation.



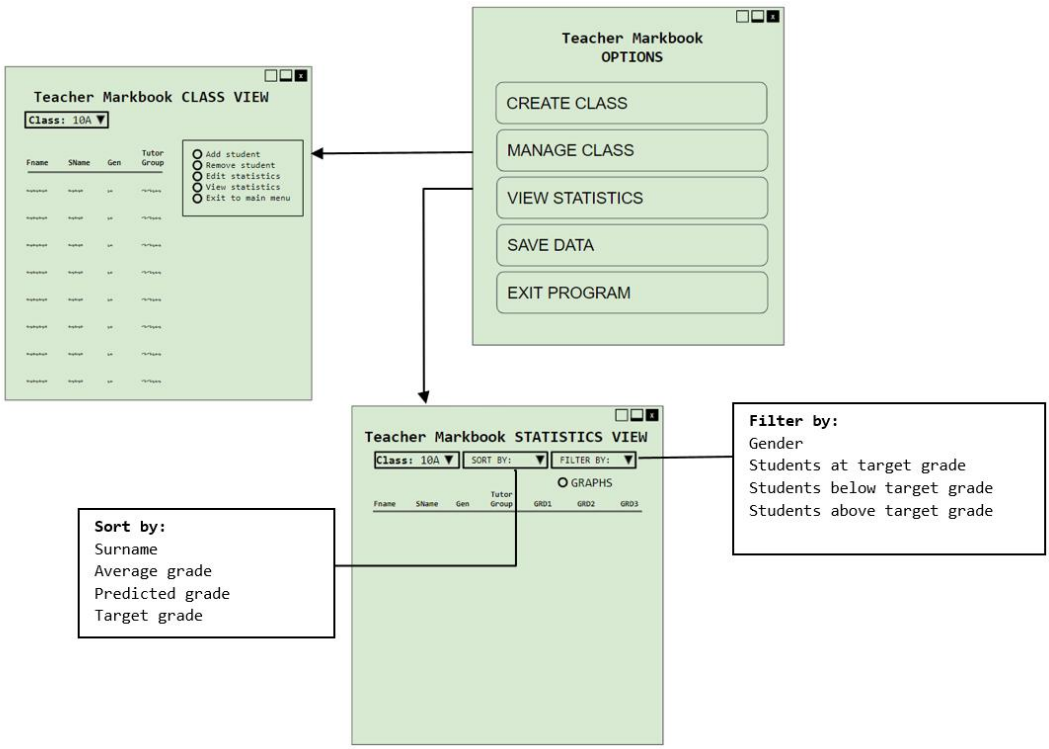

```
Here is flowchart showing the menu screen for
markbook.

Each option in the menu calls the appropriate
function.  I have structured the program this
way because it allows me to work in a modular
fashion, making my program easier to debug.  I
can also complete and test each of the
individual options separately without having
to make them work within another part of the
program.
```

© MrBComputingStuff 2022 - 2025

Here is an example IPSO diagram for the markbook program – this IPSO diagram represents what inputs/processes/storage/outputs for the `Create test` subroutine. A justification would also need to explain the reasons for structuring the system in this way.

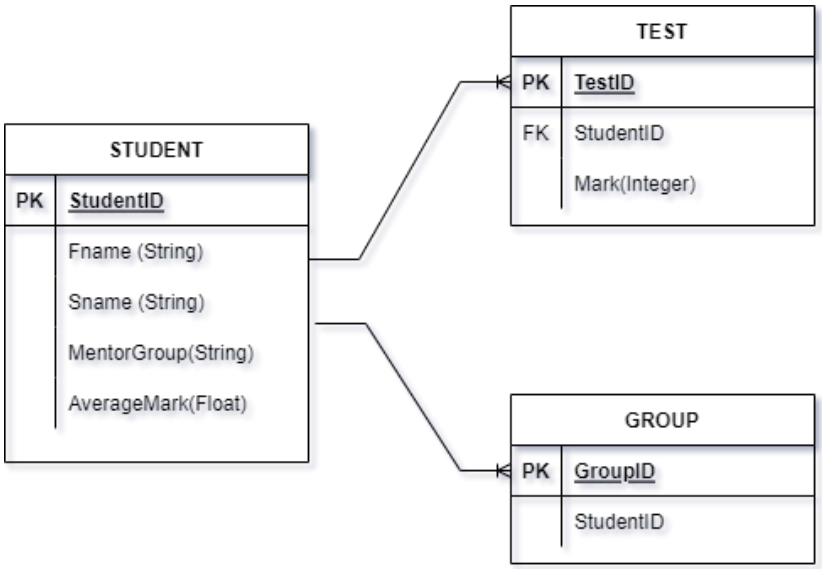
Create test			
INPUT	**PROCESS**	**STORAGE**	**OUTPUT**
Test name Max mark Grade boundaries	Append information to the TESTS list	Append list to file	On-screen summary of the test created (i.e. name, max mark, grade boundaries)
Button press for on-screen menu on the Create test screen: New test, edit test, delete test, back to home	Call relevant subroutine, i.e.: ▪ Newest() ▪ EditTest() ▪ DeleteTest() ▪ Home()	n/a	Appropriate subroutine launched

Here is an excerpt from a diagram representing the all the parts of the markbook project. Note the student would ALSO need to write an explanation justifying their reasons for structuring the program in this way.



© MrBComputingStuff 2022 - 2025

Entity Relationship Modelling (ERM) is the ideal way of showing how a database system could be structured (as with all the other methods of showing system structure, you would need ensure that you explain your choices)



CHOOSE THE RIGHT TOOLS FOR THE JOB

If you are going to produce flowcharts, consider using an application that will make this process easier. Not all application programs suit all diagram types. *For example, while applications like MS Word and PowerPoint contain flowchart symbols, they aren't really suited to creating this kind of diagram in an intuitive way.*

- **Flowgarithm** is a free MS Windows program that allows you to build (and even run) flowcharts
- https://app.diagrams.net/ is a web-app which allows you to build flowcharts, entity relationship diagrams, OOP class diagrams and the like.
- **LucidChart** is another web-app specifically designed for creating diagrams. Functionality is limited to THREE editable diagrams in the free version but this should be enough for your needs.

Please note - obviously there is no guarantee that these applications will be up and running when you have need of them, but at the time of writing they are available and work very well.

© MrBComputingStuff 2022 - 2025

3.2.2 b)
Describe the parts of the solution using algorithms justifying how these algorithms form a complete solution to the problem

What you should do in this section:

- **WRITE** an algorithm (not code!) representing each part of the solution mapped out on your decomposition diagram
- **EXPLAIN** how each algorithm contributes towards the whole solution

Every part of the decomposition diagram you produced for 3.2.1a needs to be represented algorithmically. Hopefully, you can see why it is so important that to ensure that you have broken down your problem as much as possible. While it may mean that will need to write quite a few algorithms, it easier to write a large number of smaller, less complex algorithms, then it is to write fewer, more complex algorithms for a system that has not been broken down effectively!

It is highly recommended that you represent your solution as pseudocode as opposed to a flowchart (though flowcharts are acceptable)

If you are using OOP to produce your project, you should produce an algorithmic representation of each method that you identified when you designed your classes.

The example below shows how an algorithm can be written in pseudocode with an appropriate written justification accompanying it. Note the diagram at the foot of the page that shows how the subroutines might relate to each other.

```
def PlayerMove(xPos,yPos):
        events = pygame.event.get()
      for event in events:
            if event.type == pygame.KEYDOWN:
               if event.key == pygame.LEFTKEY:
                  direction = 4
               elif event.key == pygame.RIGHTKEY:
                  direction = 2
               elif event.key == pygame.UPKEY:
                 direction = 1
               elif event.key == pygame.DOWNKEY:
                  direction = 3
               else:
                  direction = 0
            if direction != 0:
               OKtoMove = CollisionCheck(self, sprite,
            direction, Xpos,Ypos)
            if OKtoMove == TRUE:
               if direction == 4:
                      xPos = xPos -1
               elif direction == 2:
                      xPos = xPos +1
               elif direction == 1:
                      yPos = yPos + 1
               elif direction == 3:
                      yPos = yPos - 1
```

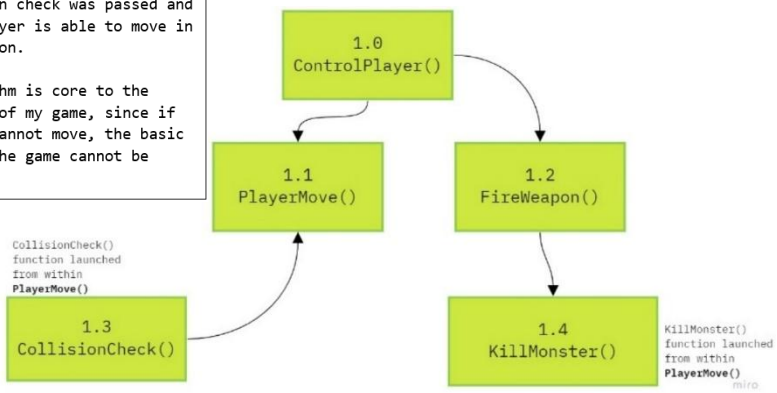
1.1 Player Movement

This function makes use of the Pygame event handler to check whether the player has pressed any of the movement keys. If a non-movement key is pressed the input is ignored.

Before the player's character is moved in the relevant direction, the **CollisionCheck** function is called, and its result stored in **OKtoMove**.

This is a separate function since I am trying to ensure that my game is developed using a modular approach and it enables me to If **OKtoMove** is true, it means that the collision check was passed and that the player is able to move in that direction.

This algorithm is core to the functioning of my game, since if the player cannot move, the basic premise of the game cannot be carried out.



3.2.2 c)
Describe usability features to be included in the solution.

What you should do in this section:

- **DESIGN** a GUI for every part of your system
- **CONSIDER USABILITY** (and where appropriate, accessibility) and how you can make your system as easy-to-use as possible
- **ANNOTATE** your designs to explain how you have implemented these ideas

When we think about usability, we are considering the user's "hands-on" experience with your program. You will need to design and implement a **GUI** (Graphical User Interface) that will make it possible (and straightforward!) for your user to interact with the program. OCR make it clear that a GUI is required:

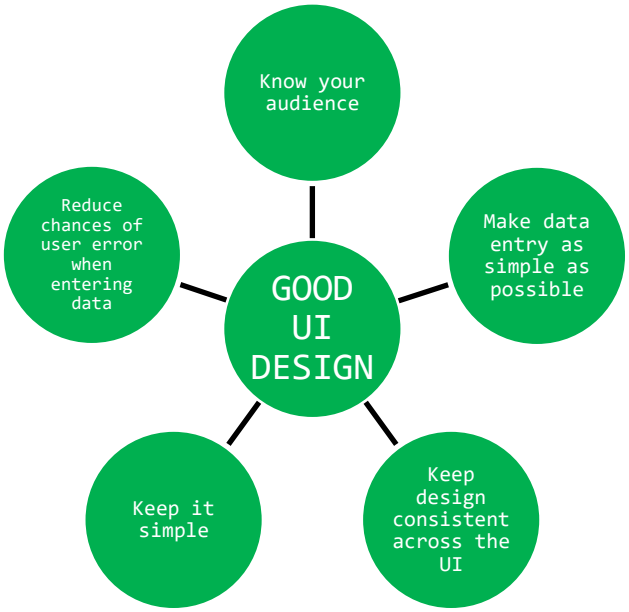
*"**It is a requirement of the specification that candidates create an appropriate GUI**. However, there are rare circumstances where this might not be appropriate. Should a candidate wish to create a program that is more suited to a command line interface then you must receive prior authorisation for this from OCR. "* **OCR Project Setting Guidance Page 4**

A **Graphical User Interface** could include a combination of a number of features, for example:

- **Windows** (Resizable? Options to minimise/maximise/close the window)
- **Toolbars/Menus** (A list of key commands at the top of a window, each with its own menu of additional options)
- **Radio-buttons** (used to select an option from a list)

You should look back at your analysis to see if the user has any accessibility issues that can be addressed through your GUI design. (*For example, catering for sight impairments and colour blindness*)

You will need to create (and therefore design) a GUI for every part of the system. You may find it helpful to explore some of the research about the key ideas of effective design…



© MrBComputingStuff 2022 - 2025

On the right is a QR code to an excellent blog post on Adobe's website, written by designer Nick Babich. The blogpost**, "The 4 Golden Rules of UI Design"** is highly recommended, containing as it does, ideas from a range of well-regarded experts in this field.

The second QR code links to another site with some excellent ideas on the absolute basics of UI design. This is another good starting point.

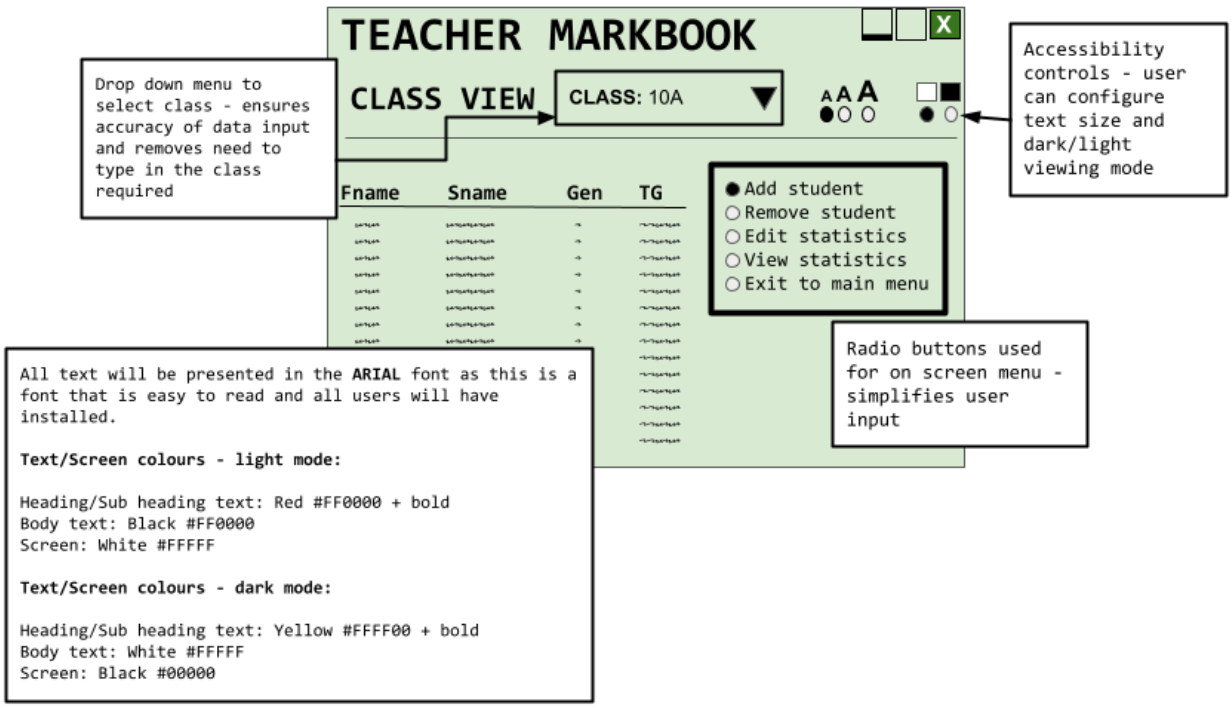
REMEMBER - You are not being assessed on artistic merit – your designs just to make clear:

a) What elements will appear on each screen
b) What those elements look like
c) What each element is for

While you could use a proper graphics package such as **Photoshop or GIMP** to make your screen designs, it is recommended that you use a tool like **Google Slides or PowerPoint**.

These programs are easy to use and will have all the features you are likely to need, including inbuilt libraries of simple shapes. You can also ensure consistency by creating a template for a UI screen and copying this for each additional screen in your program. That way, you can ensure that the elements that need to appear on each page will look the same – remember that consistency is an important part of GUI design.

Below is a screen design for part of the teacher markbook project. This was produced using Google Slides



3.2.2 d)
Identify key variables / data structures / classes justifying choices and any necessary validation.

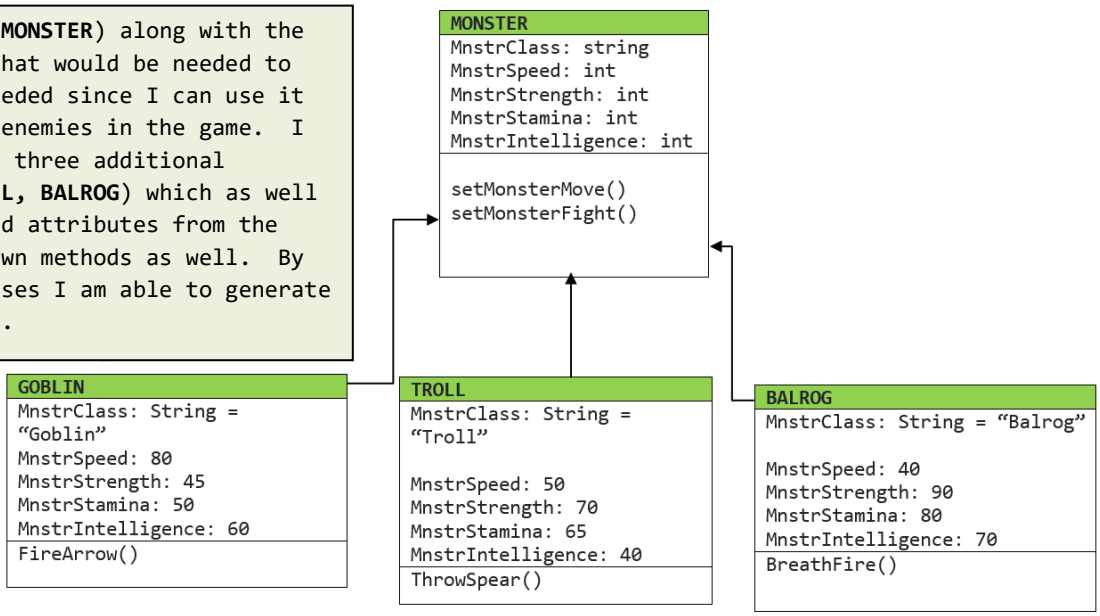
Now that you have started to plan each module of your solution, you will need to plan the main variables and data structures that your program will use. If you are using OOP, you will also need to consider the classes /subclasses /attributes /methods that will be defined in your program.

What you should do in this section:

- **IDENTIFY** all the MAIN variables and data structures that will need to be defined in each section of the program
- (For OOP solutions) IDENTIFY the classes, subclasses and attributes needed in each section of the program
- JUSTIFY why each of the above is necessary, keeping in mind the objectives of your system

An example of creating, representing (and explaining) classes & subclasses:

I have created a class (**MONSTER**) along with the attributes and methods that would be needed to run it. The class is needed since I can use it as template for all the enemies in the game. I have used this to create three additional subclasses (**GOBLIN, TROLL, BALROG**) which as well as inheriting methods and attributes from the base class, have their own methods as well. By making use of these classes I am able to generate enemies more efficiently.



The attributes in each class/subclass could be explained as in the table below:

ATTRIBUTE	TYPE	USAGE	VALIDATION	REASONS FOR INCLUSION
MnstrClass	STRING	Identifies the "class" of monster	Text	All monsters need to be named so
MnstrSpeed	INT	Dictates the maximum movement speed of the monster	0-100	Monster speed is a good way of ensuring that the player will need to adopt different tactics against different monsters.
MnstrStrength	INT	Dictates the amount of damage that the monster can do in combat	0-100	Different monsters need to have different combat abilities in order to ensure appropriate challenge within the game
MnstrStamina	INT	Dictates the amount of damage the monster can take before it dies	0-100	This is again important when trying to ensure that some monsters are harder to defeat than others
MnstrIntelligence	INT	How effectively they are able to hunt the players character	0-100	Will be used as a factor in the pathfinding algorithms to ensure that some monsters are "smarter" than others

© MrBComputingStuff 2022 - 2025

Each subroutine could have a table of its own, to record the variables and data structures that it will need. Bear in mind that some of your variables might be **GLOBAL** and therefore used in several places in your program. Global variables only need to be explained once – but do ensure that you record each place in the program in which they will be used.

Below is an example of how the variables used in one of the dungeon crawler subroutines could be explained:

The name of the variable /data structure

The data type of the variable

The module of the program to which the variables belong

Explanation of what the variable is for

Which values are valid and should be accepted by the program?

Justification needed for MB4 – You need to explain why each variable is required

PlayerMove

Identifier/data structure	Data type	Purpose	Validation	Justification
Xpos	integer	Stores the players horizontal position on the screen	0 - 640	Without this variable it would not be possible to set or change the horizontal position of the plater
Ypos	integer	Stores the players vertical position on the screen	0 - 480	Without this variable it would not be possible to set or change the vertical position of the player
event	string	Stores the result of GetKey when the list of pending events is returned	Any value	Build into PyGame and this variable is required when handling events
OKtoMove	Boolean	This is returned by the COLLISIONCHECK function and will determine whether the player can move in the direction that corresponds with their key press	TRUE, FALSE	Player movement cannot be unconditional – with this value (set by the collision check) the player would always move when the corresponding key was pressed and could pass the through walls, objects etc.
Direction	Integer	Value represents the direction that the player will move – up, down, left, right – the value of direction is checked in a selection statement that handles movement	1,2,3,4	The direction variable is necessary since it is used with selection statements to determine which direction the player will move in. The correct selection statement must be chosen based on the value of this variable.
UPKEY	String	Stores the current key for "up" as set by the player	Selected by the player but must be unique	Each of these variables is necessary as the control keys can defined by the player and without a variable to store the value of the key used for each direction, this would not be possible.
DOWNKEY	String	Stores the current key for "down" as set by the player		
LEFTKEY	String	Stores the current key for "left" as set by the player		
RIGHTKEY	String	Stores the current key for "right" as set by the player		

It is only necessary to reference the *key* variables that you will use. *For example, if your program uses a FOR loop which is set to run x times, you may only need to write about the variables used INSIDE the loop that might perform more important tasks, if the purpose of x is just to control the loop.*

© MrBComputingStuff 2022 - 2025

3.2.3 DESCRIBE THE APPROACH TO TESTING

"Discovering the unexpected is more important than confirming the known"

George E P Box

While statistician and mathematician George Box wasn't thinking about software when he wrote the quote above, it is highly relevant to the idea of software testing.

You will obviously become very familiar with your project throughout its development, and this is the danger with testing – it is easy to miss errors in your program when you think you know where all the errors are likely to occur!

It is also a good idea to be clear about what we mean by testing. You need to carry out two types of testing:

- **Iterative testing**
- **Post development testing**

	Iterative testing (WHITE BOX TESTING)	Post-development testing (BLACK BOX TESTING)
Takes place.	-Throughout the development of the system	-After the system has been developed
Tests.	-That all sections of the code work correctly -That code contains no syntax, logic or runtime errors.	-That the solution performs as expected -That the solution meets the requirements laid out in the ANALYSIS -That the solution meets the expectations of stakeholders

Both forms of testing might identify different errors and issues in your system. You should ensure that you plan your tests carefully to intercept all the different error types that you may encounter. You will also need to ensure that you choose appropriate data with which to carry out these tests.

Syntax errors	Errors which prevent the program from compiling (*i.e. misspelt keywords, missing parentheses*)
Logic errors	The program compiles and runs, but generates unexpected outputs *(i.e. a subroutine for calculating the average of 5 grades uses correct programming syntax but contains the wrong formula for calculating an average)*
Runtime errors	The program is syntactically correct (it compiles and runs) but an expected event causes it to crash *(for example, a divide by zero error)*

Valid data	Data that the system should accept
Invalid / erroneous data	Data that the system should reject
Extreme / borderline data	Data on the "edges" of the valid data (for example, if the valid data is between 1 and 100, you would test that the system would accept 1 and 100

© MrBComputingStuff 2022 - 2025

Remember that iterative means "repeating" - this means that you must test your program THROUGHOUT its development and use the outcomes to inform the development of your program.

Data validation

This can be done during both iterative and post development testing. Users should be able to test that data validation works simply by using the program. (i.e. through the need to enter data into the system)

Validation	Example
Range check	That data falls within an appropriate range – for example, a number between 1 and 10 is required
Presence check	That the field has not been left blank if data entry is required
Length check	That the length of the data does not exceed a given value; for example, if a password must contain 10 characters or less
Format check	Checks that data has been entered in the correct way; for example, a data must be entered in the format DDMMYY
Type check	Data must be of a specific type; for example, a name cannot contain numeric data

A good programmer will design the system in such a way that data entry is simplified as much as possible to remove the need to create complex validation processes. For example, using **input sanitisation** makes data entry more efficient for the user and also reduces the likelihood of errors occurring in this process.

Post development testing

It is a good idea to involve your stakeholders in your post development testing.

End users may well find errors or issues with your program that you would never have considered. Remember that your stakeholders will be dealing with a finished program so you will need to approach the testing in a different way to the iterative testing.

You should not expect users to have the expertise or knowledge needed to test code and so you will need to provide them with test scenarios that you can ask them to run.

For example, here is a simple scenario that could be used to test player movement and collision detection.

The player should attempt to move their character to the exit of the level.

They should attempt to collide with walls.

The collisions should occur from all four directions (above, below, from the left, from the right)

They should attempt to move onto the exit:

a) Without collecting key
b) After collecting the key

This kind of testing would not be possible when carrying out iterative testing (at least to begin with) since it requires a functioning game.

© MrBComputingStuff 2022 - 2025

3.2.3 a)
Identify the test data to be used during the iterative and post-development phases, and justify the choice of this test data

What you should do in this section:

- **WRITE** a test plan (including a range of test types) for iterative testing
- **WRITE** a test plan (including a range of test types) for post-development testing
- **JUSTIFY** your reasons for including each test

Here is an example test table for the markbook project. Here, the programmer has carried out tests using:

- **valid data**
- **invalid data**
- **borderline data**

Note that two of tests at least partially fail. The programmer would need to explain what action would be taken as a result. In addition, the programmer would need to ensure that tests are planned that would spot possible logic errors – *for example, testing that a range of values entered into the system would calculate an average correctly.* Also note that the test plan **SHOULD NOT contain actual evidence of testing at this point.** It is, after all, impossible to test a program before it has been developed!

Explain what the test will do

What data you will use to perform the test

What should happen when the test is carried out

What the actual outcome of the test was

Cross reference to the evidence that you have for the outcome of the test

Justification for including the test

Test number	Description	Test Data	Expected outcome	Outcome	Evidence	Reason for including this test
1	User enters appropriate value for test mark	Letters, numbers, characters	Any value other than numbers is rejected	PASSED	Screenshot 1	I need to make sure that if the user enters anything other than a number for the test mark, the input is rejected
2	Check that the marks on edge of the valid range are accepted	1, 100	Both values should be accepted	FAILED (100 was rejected)	Screenshot 2	I need to make sure that the user can enter the lowest possible and highest possible mark without these values being rejected
3	Check that marks outside the valid range are not accepted	-1,101	Both values should be rejected	PASSED	Screenshot 3	This test is important because I need to ensure that the user cannot enter test marks outside the range of valid values
4	Check that numbers within the accepted range are accepted	20, 50, 64.8	20 and 50 should be accepted 64.8 should be rejected	FAILED (64.8)	Screenshot 4	I also need to make sure that acceptable values are not rejected by the program, or it will not meet the success criteria

© MrBComputingStuff 2022 - 2025

Here is an extract from a test plan for the post development testing of the markbook project.

Test number	Test	Test description	Expected result	Reason for including this test
1	ADDING STUDENTS TO THE SYSTEM	Add student data to the system and save file. The new data should be there when the program is re-opened	-User can enter data appropriately with invalid responses rejected -Data is saved successfully to file	Adding students to the system is a basic requirement of the system. It is important to test not only that the data can be added, but that it will be saved to file properly and be present when the program is used next.
2	REMOVING STUDENTS FROM THE SYSTEM	Locate a student in the system; the data for this student should be deleted and the file updated appropriately.	-User data retrieved if it exists in the system -Data is removed from the system -Data file is updated appropriately	As above, removing students from the system is basic functionality. It is important to test both the data handling within the program the file handling after the data is deleted.

© MrBComputingStuff 2022 - 2025

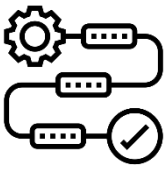
OCR A Level Computing
Unit 3 NEA

STUDENT GUIDE
PART 3

AO 3.2 Developing the coded solution (maximum 25 marks)			
Iterative development of a coded solution (maximum 15 marks)			
1–4 marks	**5–8 marks**	**9–12 marks**	**13–15 marks**
The candidate will have:			
• Provided evidence of some iterative development for a coded solution. • Solution may be linear. • Code may be inefficient. • Code may not be annotated appropriately. • Variable names may be inappropriate. • There will be little or no evidence of validation. • There will be little evidence of review during the development.	• Provided evidence for most stages of the iterative development process for a coded solution describing what they did at each stage. • Solution will have some structure. • Code will be briefly annotated to explain key components. • Some variable and/or structure names will be largely appropriate. • There will be evidence of some basic validation. • There will be evidence that the development was reviewed at some stage during the process.	• Provided evidence of each stage of the iterative development process for a coded solution relating this to the break down of the problem from the analysis stage and explaining what they did at each stage. • Provided evidence of some prototype versions of their solution. • The solution will be modular in nature. • Code will be annotated to explain all key components. • Most variables and structures will be appropriately named. • There will be evidence of validation for most key elements of the solution. • The development will show review at most key stages in the process.	• Provided evidence of each stage of the iterative development process for a coded solution relating this to the break down of the problem from the analysis stage and explaining what they did and justifying why. • Provided evidence of prototype versions of their solution for each stage of the process. • The solution will be well structured and modular in nature. • Code will be annotated to aid future maintenance of the system. • All variables and structures will be appropriately named. • There will be evidence of validation for all key elements of the solution. • The development will show review at all key stages in the process.
Testing to inform development (maximum 10 marks)			
1–2 marks	**3–5 marks**	**6–8 marks**	**9–10 marks**
The candidate will have:			
• Provided some evidence of testing during the iterative development process.	• Provided some evidence of testing during the iterative development process. • Provided evidence of some failed tests and the remedial actions taken.	• Provided evidence of testing at most stages of the iterative development process. • Provided evidence of some failed tests and the remedial actions taken with some explanation of the actions taken.	• Provided evidence of testing at each stage of the iterative development process. • Provided evidence of any failed tests and the remedial actions taken with full justification for any actions taken.

© MrBComputingStuff 2022 - 2025

3.3.1 Iterative development process



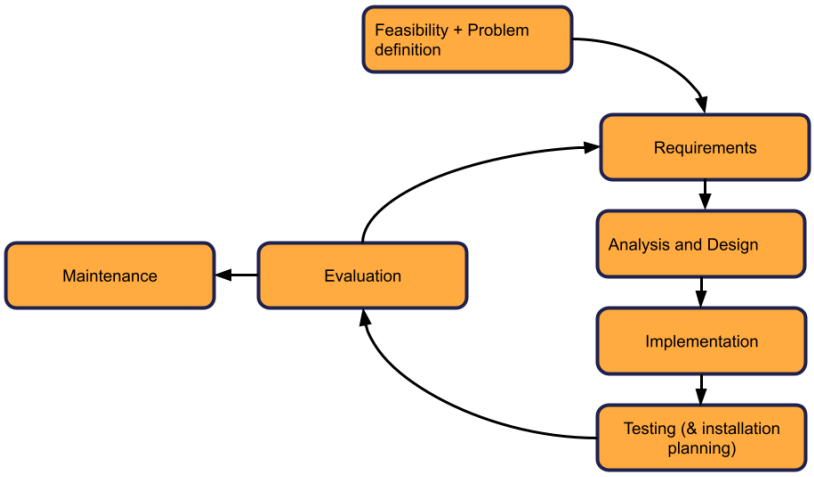
"Don't comment bad code – rewrite it"

Brian Kernighan

As the inventor of the UNIX operating system, Brian Kernighan knows a thing or two about working on something again and again until it is good enough to release. It is inevitable that you will need to return to some sections of your project over and over again. Getting it to work is one thing, getting to work WELL is something else entirely.

Remember that the development process is SUPPOSED to be iterative – i.e. you should expect to write, test and modify your code a number of times over. As part of your analysis you will have identified and spoken to stakeholders, who will have given you important insights as to what your finished project should look like. It is important to return to your stakeholders during the development process.

The AGILE development model



You will be following a working process that is based around the Agile development methodology.

You will need to develop and test a number of prototype solutions.

Each solution will be tested and improved in an iterative manner, with stakeholders involved with the development of prototype.

3.3.1 a)
Provide evidence of each stage of the iterative development process, justifying any decisions made

3.3.1 b)
Provide annotated evidence of prototype solutions, justifying any decisions made

What you should do in this section:

- Provide **EVIDENCE** that you have taken an **ITERATIVE** approach to development
- **SHOW** what development has occurred in each iteration
- **EXPLAIN** and **JUSTIFY** what you have done and why
- **TEST** and gather **STAKEHOLDER FEEDBACK**
- Set targets for the next iteration of your work

You don't need to treat 3.3.1a and 3.3.1b as separate sections of the coursework.

There is nothing stopping you from doing so but you might find it easier to structure and present your evidence if you approach these spec points as a single piece of work.

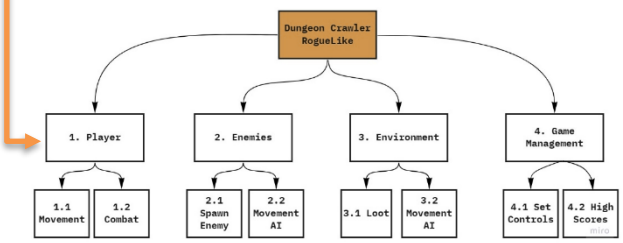
Evidencing the development process

It is easy to get confused in this section unless you approach it in a structured way. You might want to consider tackling each iteration of your solution in the same way so that you can take a consistent approach to evidencing it. *For example, you are encouraged to take a modular approach; refer back to your decomposition diagram and tackle each of the modules of your program in turn*.

Below is a SUGGESTED method of presenting and evidencing the different iterations of your work:

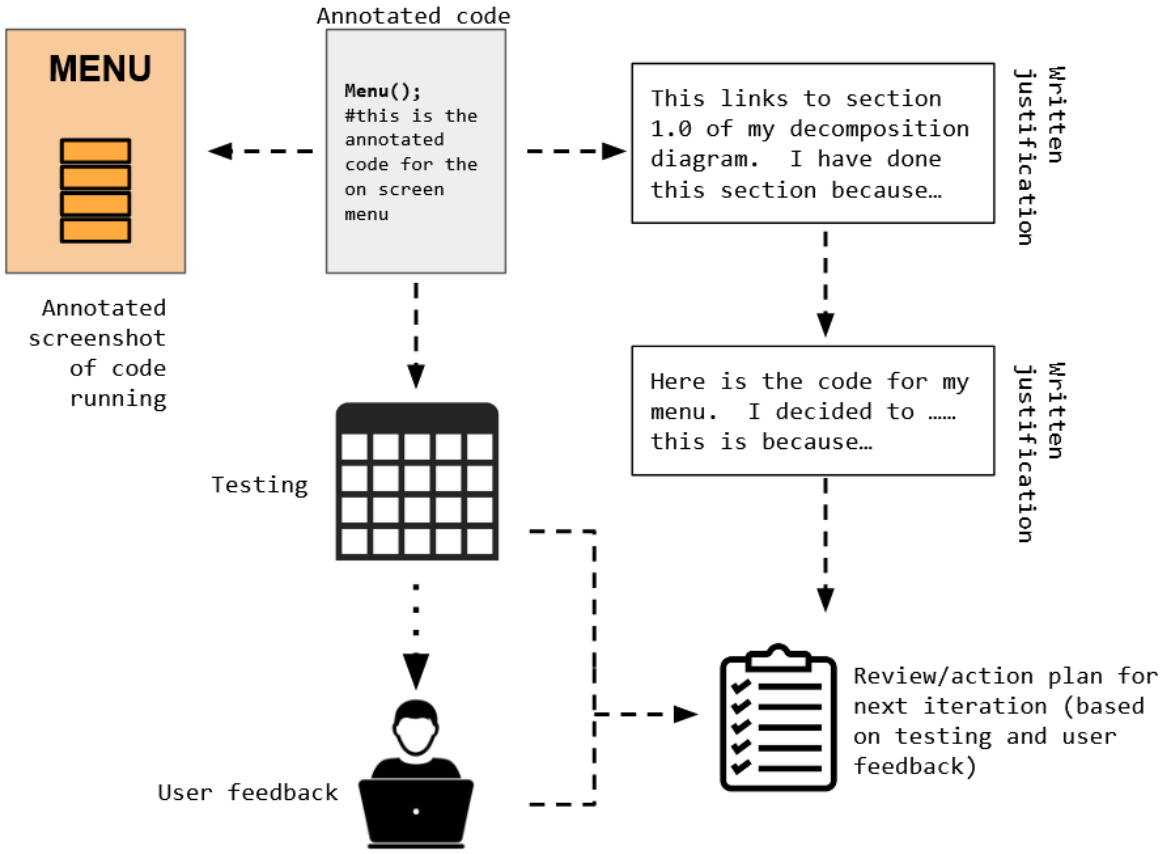
1.0 Player

TASK	EVIDENCE BY
Write code	**Extract of code (annotated)**
Evidence your code running	**Screenshots/Video**
Relate code back to your decomposition	**Written explanation**
Justify the decisions you took in completing this code	**Written explanation**
Test the code and report on any issues (such as failed tests)	**Test table/written explanation**
Obtain Stakeholder feedback	**Written statement/interview**
Review/identify next steps for next iteration	**Written explanation**



The diagram below shows this possible way of evidencing iterative development for your solution.

However, as long as it is clear that you have tackled the solution iteratively and taken care to evidence each prototype accordingly, with justification for choices you have made, the method you choose to do so is not important.



On the next slide page is an example of what the development part of your work might look like. It is annotated to include key indicators of things which are likely to gain you higher marks. Remember that this is a suggestion only – though students who have used this or a similar approach, generally have tended to do well.

© MrBComputingStuff 2022 - 2025

Show that you have developed your solution iteratively – indicate which iteration of which section you are working on

Stage 1, version 1.0

Moving the player character

This is my first attempt at the game code to move the player sprite. I have assigned the arrow keys as controls for this version, just for testing purposes. The user will be able to choose their own controls in my final version, as this was specified as a user requirement. I have set the sprite as a rectangle for now and will add a permanent one later – it doesn't matter what it looks like for now, so long as the code works

```
keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        sprite_x -= sprite_speed
    if keys[pygame.K_RIGHT]:
        sprite_x += sprite_speed
    if keys[pygame.K_UP]:
        sprite_y -= sprite_speed
    if keys[pygame.K_DOWN]:
        sprite_y += sprite_speed

    # Draw the sprite
    pygame.draw.rect(screen, BLUE, (sprite_x,
sprite_y, sprite_size, sprite_size))

    pygame.display.flip()
    clock.tick(60)
```

Show the code that you added for this part of your project, along with annotation/explanation

Test the code, using the relevant tests from the test plan you drew up in the design section. Some students highlight the tests red or green to denote passing or failing (as here)

TEST	DESCRIPTION	TEST DATA	EXPECTED OUTCOME	OUTCOME	REASONING
#1	Player sprite responds appropriately to movement keys	User input (arrow keys)	Player sprite moves in the correct direction	Error message generated (see screenshot #1)	NameError generated

Notice the cross referencing to the relevant screenshot evidence.

Refer to the relevant success criteria identified in your analysis

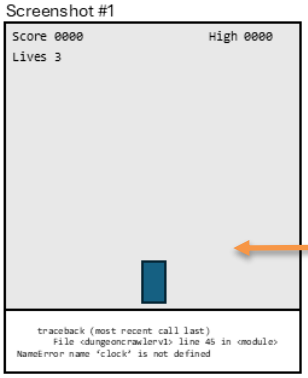
LINK TO SUCCESS CRITERIA		
CRITERIA	EVIDENCE	JUSTIFICATION
#1 Game must support user defined controls	- Screenshot showing that keys have been successfully remapped	This is an important criteria because it ensures that both left and right-handed players can play the game
#2 Player sprite responds appropriately to chosen movement keys	- Player feedback - Videos	This is important because stakeholders identified responsive controls as an important factor in what makes a game enjoyable

Here is where a review of this part of the program takes place. Notice that the error in the code is discussed and the corrected code is evidenced.

Review

As you can see from screenshot #1, the program did not run. A NameError code was generated, and I realised that I had not defined the 'clock' variable needed to set the frames per second movement variable for the sprite. I added this variable to the program to address this problem (see below). I have not yet met either of the success criteria relevant to this part of the program, though I am not concerned given that this is only the first version and I can now complete them, having fixed the error.

Screenshot #1



The code is tested again after the error is fixed – the green formatting shows that it has passed

```
clock = pygame.time.Clock()
keys = pygame.key.get_pressed()
```

Where possible and where appropriate, involve your stakeholders in each prototype. You may not need them be involved with every stage of the development. *In this example, it was a good idea to obtain user feedback because the code was linked to the player controls.*

TEST	DESCRIPTION	TEST DATA	EXPECTED OUTCOME	OUTCOME	REASONING
#1	Player sprite responds appropriately to movement keys	User input (arrow keys)	Player sprite moves in the correct direction	Error message generated	Clock variable had not been defined

Your review should also show that you have a clear idea of what you need to do next – and why.

Stakeholder feedback

With the error fixed, stakeholders were happy with the responsiveness of the controls. I made sure that they understood that the controls set in the prototype were for test purposes only and that the rectangle was a temporary sprite.

Next steps

Now that the basic code works, I will need to replace the rectangle with a character sprite. I also need to add the facility for players to choose their own controls. I will make these changes in the next iteration of my program. I will also be able to add the sprites for the enemy into the game using similar code now that I have successfully made the player move, This will also be something that I will do in version 2.

© MrBComputingStuff 2022 - 2025

Some tips / points to note.

- All screenshots should be clear (i.e. readable) and should be annotated.
- Video evidence can be used (for example, recording of an interview, recording of a program in operation) but you need to ensure that video is made accessible to the marker and the moderator. **Moderators are told not to click on links that students might include in their work.** This means that your video evidence must be included with your work – ***you cannot upload it to the cloud and provide a link to it within your work.***

- You are advised to submit all video evidence on a memory stick (if your school does postal moderation) or – or upload it all to a folder in cloud storage service (i.e. Google Drive) if your school submits the work online. Ask your computing teacher which method your school uses. If your school submits the work online, your teacher will need to upload every video file along with the work itself – so you MUST include each one.

- Each video file needs to be clearly named, with the filename including your name as well as your centre and candidate number.

- The nature of modular programming means that you may not be able run some modules independently of each other – you should evidence and test each one as best you can.
- In addition, it may not be possible (or appropriate) to gather stakeholder feedback at every stage in the process.
- Some prototypes might feature very little development between iterations – there may be only a small section of code that actually changes. In this case, you need only feature the parts that have changed.
- Including some form of review at the end of each iteration is very important. You should examine the progress you made in the current iteration and explain what your aims are for the next iteration of your program.

Annotating code

It is important to annotate your code… but you should remember the *purpose* of annotation:

*"Annotation is maintainability technique used by programmers. It involves adding **useful** and **relevant** comments to the code so that the programmer – or anyone else - can make changes to it later. "*

When you are annotating code there are few things to bear in mind:

1. **You shouldn't have to explain variable names.**

```
x = 0
#create a variable (x) to store the
average of the test marks
```

Variable names should be obvious and should be representative of their contents. If you have to use annotation to explain the purpose of a variable, then you have named it appropriately.

2. **Subprogram names should not require annotation.**

```
def createNewTest();
#create a function to create a new test
```

Similarly, functions and subroutines should be named so that their purpose is evident.

It is not necessary to annotate subprograms that already have self-explanatory identifiers – and if the purpose of the subprogram is not clear from its name, then it should be changed. You may need to annotate code WITHIN the sub-program – but it should not be necessary to explain the naming.

3. **Don't annotate "obvious" code.**

Some lines of code are self-explanatory and should not require any annotation. For example, it is clear what the purpose of print statements are - labelling them is pointless and provides no benefit to the reader.

```
#print name of program
print ("**TEACHER MARKBOOK**)
print ()
CorrectChoice = False
#print choices
print("Enter your choice")
print("1. Create Class")
print("2. Delete Class")
print("3. Create Test")
print("4. Delete Test")
print("5. View Data")
print("6. Save Data")
print("7. Quit program")

while not CorrectChoice:
```

3.3.2 Testing to inform development

"It's not at all important to get it right the first time. It's vitally important to get it right the last time"

Andrew Hunt and David Thomas

You will recall that we looked included testing in the previous section of development section, as part of your evidence of iterative development. We will look at how this testing should be presented.

You will need to test your code throughout the development process and evidence this.

3.3.2 a) Provide annotated evidence for testing at each stage justifying the reason for the test

What you should do in this section:

- **Present a testing table** for each iteration of the development process
- Ensure that the purpose of each test is **justified**
- Include **annotated evidence** detailing the outcome of the test

You should already have planned testing as part of your design. It is unlikely that you can test EVERYTHING in each iteration, but you should ensure that everything on your test plan is tested by the time your project is complete.

Your IDE may have "debugging" features – in which case it would be well worth exploring the use of these for your testing.

© MrBComputingStuff 2022 - 2025

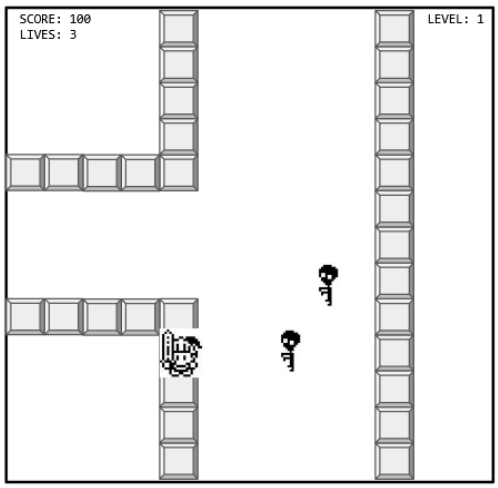
FEATURE	USAGE
BREAK POINTS	The program will cease execution at a point determined by the program (useful if you want the program to stop before it reaches a section that you know doesn't work properly)
VARIABLE WATCH	Will display the current value of a chosen variable
TRACING	The value of variables is logged and displayed as the program is executed
MEMORY INSPECTOR	Allows the programmer to see what is in a particular section of memory at a given time – useful if there are memory leaks or you program is returning an "out of memory" error

The evidence of the testing is given as screenshot #1 and #2. It is much better to cross reference the screenshots with the table but to include them separately – this ensures that they are readable and that you have the space to annotate them properly

Here is an extract from a test plan for the roguelike game.

Test number	Description	Test Data	Expected outcome	Outcome	Evidence	Reason for including this test
1	Test to see if collision detection works when player touches wall	Value of **PlayerCollision** Boolean variable	Value of **PlayerCollision** is True (Player cannot move through walls)	FAIL	Screenshot #1 & #2	I need to ensure that the player's character cannot move through scenery such as walls, or the game will be unplayable

Screenshot #1



The screenshot here shows the result of my failed test. You can see the player character has moved over the top of the wall. The collision detection should prevent this from happening.

Screenshot #2

```
def checkCollision(self, sprite1, sprite2):
    col = pygame.sprite.collide_rect(sprite1, sprite2)
    if col == True:
    PlayerCollision = True
    sys.exit()
```

VARIABLE WATCH

NAME	TYPE	VALUE
PlayerCollision	Boolean	False

I have examined the code that runs when the collision detection routine runs, using a variable watch to check the value of **PlayerCollision**, the Boolean which us set to True if a collision has taken place. As it is still set to False, I can see there is a problem with this subroutine. I need to look at this code again as there is clearly something here which is preventing the collision detection from working

Evidence of testing here has been shown with screenshots of the gaming running, as well as annotated code, with a variable watch placed on the variable that programmer believes is causing the error.

© MrBComputingStuff 2022 - 2025

3.3.2 b)
Provide annotated evidence of any remedial actions taken, justifying the decision made

What you should do in this section:

Show that you have addressed actions identified in any failed tests by:

- Providing **annotated evidence** of the corrected code
- Providing **annotated evidence** of the code running
- Providing a **short, written explanation** that **justifies** the actions you took

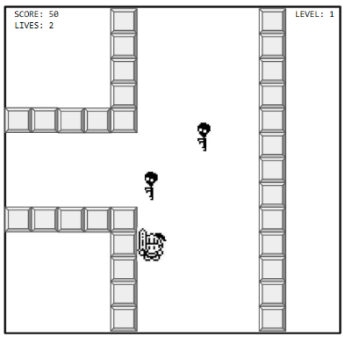
This is a straightforward section to complete since you are following up any failed tests that were carried out during your iterative development process.

The amount that goes into this section will depend on how many of your tests passed or failed. Just like with the other work you have done, proper evidencing is required, and you need to justify the reasons for taking the corrective actions that you did.

Below we can see how the programmer has addressed the problem with collisions, shown in the previous example.

- **The code has been tested again and evidence of the results provided.**
- **There are annotated screenshots of the changes, each with a suitable explanation/justification.**

Screenshot #5



The screenshot here shows that my character is now unable to collide with walls – the changes to my code have corrected the collision detection

Screenshot #6

```
def checkCollision(self, sprite1, wallsprite):
    col = pygame.sprite.collide_rect(sprite1, wallsprite)
    if col == True:
    PlayerCollision = True
    sys.exit()
```

VARIABLE WATCH

NAME	TYPE	VALUE
PlayerCollision	Boolean	True

Here you can see the changes made to my code. The value of the PlayerCollision Boolean is now True as it should be. I realised that I was using referring to sprite2 instead of wallsprite in my original code. This meant that the code was looking for a collision with sprite2 instead of the wall, so no collision was detected. Changing this value corrected the problem.

Test number	Description	Test Data	Expected outcome	Outcome	Evidence	Reason for including this test
1	Test to see if collision detection works when player touches wall	Value of **PlayerCollision** Boolean variable	Value of **PlayerCollision** is True (Player cannot move through walls)	PASS	Screenshot #5 & #6	This test was failed in my previous iteration – it was important to address the errors my collision code if the rest of the game is to work properly.

© MrBComputingStuff 2022 - 2025

OCR A Level Computing Unit 3 NEA

STUDENT GUIDE PART 4

AO 3.3 Evaluation (maximum 20 marks)			
Testing to inform evaluation (maximum 5 marks)			
1 mark	**2 marks**	**3–4 marks**	**5 marks**
The candidate will have:			
• Provided evidence of some post development testing.	• Provided evidence of final product testing for function.	• Provided annotated evidence of post development testing for function. • Provided annotated evidence for usability testing.	• Provided annotated evidence of post development testing for function and robustness. • Provided annotated evidence for usability testing.
Evaluation of solution (maximum 15 marks)			
1–4 marks	**5–8 marks**	**9–12 marks**	**13–15 marks**
The candidate will have:			
• Commented on the success or failure of the solution with some reference to test data. • The information is basic and communicated in an unstructured way. The information is supported by limited evidence and the relationship to the evidence may not be clear.	• Cross referenced some of the test evidence with the success criteria and commented on the success or otherwise of the solution. • Provided evidence of usability features. • Identified some limitations on the solution. • The information has some relevance and is presented with limited structure. The information is supported by limited evidence.	• Used the test evidence to cross reference with the success criteria to evaluate the solution identifying whether the criteria have been met, partially met or unmet. • Provided comments on how any partially or not met criteria could be addressed in further development. • Provided evidence of the usability features. • Considered maintenance issues and limitations of the solution. • There is a line of reasoning presented with some structure. The information presented is in the most part relevant and supported by some evidence.	• Used the test evidence to cross reference with the success criteria to evaluate the solution explain how the evidence shows that the criteria has been fully, partially or not met in each case. • Provided comments on how any partially or unmet criteria could be addressed in further development. • Provided evidence of the usability features justifying their success, partial success or failure as effective usability features. • Provided comments on how any issues with partially or unmet usability features could be addressed in further development. • Considered maintenance issues and limitations of the solution. • Described how the program could be developed to deal with limitations and potential improvements / changes. • There is a well developed line of reasoning which is clear and logically structured. The information presented is relevant and substantiated.

© MrBComputingStuff 2022 - 2025

3.4.1 Testing to inform evaluation

"Program testing can be a very effective way to show the presence of bugs, but is hopelessly inadequate for showing their absence".

Edsger Dijkstra

You have provided evidence of testing during the iterative development process. Hopefully, this process – and your solution – are now complete. At this point you should test your finished solution for both **robustness** and **usability.**

You should already have a testing table ready to use for this process – you now need to ensure that you provide evidence of testing for both the aspects named above.

You will only be able to evaluate how successfully you have solved the initial problem that you defined, once you have thoroughly tested your solution.

3.4.1 a)
Provide annotated evidence of testing the solution for robustness at the end of the development process.

What you should do in this section:

- Provide **EVIDENCE** that you have tested the system for **function and robustness**
- **ANNOTATE** evidence of the testing that you carried out

Before you can begin this testing, it is important to understand what is meant for **FUNCTION** and **ROBUSTNESS**;

FUNCTION; whether a program is able to perform the functions that is has been designed to do

Testing for function is mainly concerned with ensuring that the output of the program is correct, based on the data that has been input and processing that has been carried out. You are looking to see whether validation is carried out correctly, and whether your program generates any logic errors. You might recall this information from your design…

Validation	Example
Range check	That data falls within an appropriate range – for example, a number between 1 and 10 is required
Presence check	That the field has not been left blank if data entry is required
Length check	That the length of the data does not exceed a given value; for example, if a password must contain 10 characters or less
Format check	Checks that data has been entered in the correct way; for example, a data must be entered in the format DDMMYY
Type check	Data must be of a specific type; for example, a name cannot contain numeric data

ROBUSTNESS; the ability of a program or computer system to cope with unexpected errors or situations while it is running

© MrBComputingStuff 2022 - 2025

Testing for robustness essentially means that you are trying to BREAK your program. The two most likely type of errors that will "break" your program are:

Syntax errors	Errors which prevent the program from compiling (*i.e. misspelt keywords, missing parentheses*)
Runtime errors	The program is syntactically correct (it compiles and runs) but an expected event causes it to crash *(for example, a divide by zero error)*

You need to ensure that your testing is thorough and that you have chosen appropriate test data that will provide an accurate picture of the robustness of your system. Test data could be taken from the categories below:

Valid data	Data that the system should accept
Invalid / erroneous data	Data that the system should reject
Extreme / borderline data	Data on the "edges" of the valid data (for example, if the valid data is between 1 and 100, you would test that the system would accept 1 and 100

While testing for function and usability, you will need to consider that data will come in two forms – you will need to make use both types whilst testing function/robustness.

DATA ENTERED BY THE USER	This could be data could take various forms – for example, textual data such as name, or numeric data such as test score.
DATA GENERATED BY THE PROGRAM ITSELF	This is data that might be created via an algorithm – for example, standard deviation calculated from a set of values.

How to evidence testing has already been covered in the DEVELOPMENT section of the NEA guide. Below is a reminder of the type of test table you could aim to complete in this section. Alternatively, you could opt for a style of testing similar to that which you carried out during your iterative development phase.

Explain what the test will do

What data you will use to perform the test

What should happen when the test is carried out

What the actual outcome of the test was

Cross reference to the evidence that you have for the outcome of the test

Justification for including the test

Test number	Description	Test Data	Expected outcome	Outcome	Evidence	Reason for including this test
1	User enters appropriate value for test mark	Letters, numbers, characters	Any value other than numbers is rejected	PASSED	Screenshot 1	I need to make sure that if the user enters anything other than a number for the test mark, the input is rejected
2	Check that the marks on edge of the valid range are accepted	1, 100	Both values should be accepted	FAILED (100 was rejected)	Screenshot 2	I need to make sure that the user can enter the lowest possible and highest possible mark without these values being rejected
3	Check that marks outside the valid range are not accepted	-1,101	Both values should be rejected	PASSED	Screenshot 3	This test is important because I need to ensure that the user cannot enter test marks outside the range of valid values
4	Check that numbers within the accepted range are accepted	20, 50, 64.8	20 and 50 should be accepted 64.8 should be rejected	FAILED (64.8)	Screenshot 4	I also need to make sure that acceptable values are not rejected by the program or it will not meet the success criteria

© MrBComputingStuff 2022 - 2025

3.4.1 b) Provide annotated evidence of usability testing (user feedback)

What you should do in this section:

- **DESIGN** tests for users to perform with your finished program in order to test usability
- Obtain **FEEDBACK** from your users
- **DOCUMENT/ANNOTATE** the feedback that you receive.

This part of the work requires you to perform POST-DEVELOPMENT testing and is carried out differently to iterative testing.

Post-development testing is carried out AFTER the program has been completed and is designed to see how your program performs when it is being used.

USABILITY TESTING; testing carried out by stakeholders in order to see how easy a system is to use

Usability testing has a number of objectives:

- To find out if users can carry out specific tasks **successfully.**
- To ascertain whether these tasks can be carried out **efficiently.**
- To gauge the levels of **user satisfaction**
- To identify any changes needed to **improve** the system **performance** and user **satisfaction.**
- To determine whether the performance of the system meets the **success criteria.**

As your stakeholders will be carrying out testing, you cannot expect them to be running modules of code and noting any errors that occur!

You will need to devise some scenarios, which when run, would allow the user to test the aspects of the system that you are looking at. This is hopefully something you have done already in the design section of your work.

It is important that your scenarios meet the following criteria.

1. They are clear in terms of end goal.
2. They are realistic and contain typical use cases for the system.
3. They encourage the user to interact with the interface and system as fully as possible.
4. They aren't too prescriptive and create opportunities for the users to make decision.

Below are some example scenarios that a programmer might set for usability testing. Some of them are appropriate and some of them aren't.

SCENARIO: Test that the menu works OK	Very vague scenario – it is not clear what exactly the user has to do. What does "the menu works OK" mean?
SCENARIO: Enter 30 classes and a hundred students into the system	This is an unrealistic use case and would also be an extremely lengthy scenario for the user to follow.

© MrBComputingStuff 2022 - 2025

SCENARIO:

On the menu screen click on “Add Class”. Then on the Add Class screen click on “Name Class” – Name the class “10Y Computing” “Add Student”. Enter the first name, surname, gender, mentor group and target grade. Repeat this 10times. Then click the “Save” option. Name the save file and click OK. Quit the program, using the Quit option. Then reload it, choose “Load Class” and load the class you just saved.

This scenario is too prescriptive and literally gives the user step by step instructions. How will you know how intuitive your program is to use, unless you let users make their own way through to do the tasks you set?

SCENARIO:

1. Create a new class
2. Add 10 students to the new class
3. Save the data you have entered
4. Quit the program
5. Reload the program and load the class you have just saved

This is a better version of the scenario above.

The tasks that the user needs to complete are realistic and clearly listed.

It will also require the user to find their own way through the program and will thus test how easy to use and intuitive the program is.

SCENARIO:

Generate a student report for students with an average mark of < 0

This is a good scenario, because it should not possible for a student to have an average mark of < 0. If the system is working properly the validation should prevent the user entering the criteria to generate this report.

You will need to evidence the outcomes of your testing (and what happens when users run through these scenarios) and provide appropriate commentary/annotation.

For example:

- Providing **screenshots** that show what happened when the user goes through each scenario.
- Providing **video evidence** of the user going through the scenarios, along with a commentary. Video evidence needs to be accessible to the moderator – it will be submitted alongside your NEA - so ensure that it is stored in a common video format (i.e. MP4) and that the file name contains your name/centre/candidate number:

i.e. **`eric barone 20444 8736 videoTest.mp4`**

The method you choose will partly depend on the type of stakeholders that you have and the time/level of expertise they have**. Whichever method you use, make sure that you number your tests** – this will be very important for the next part of your evaluation!

© MrBComputingStuff 2022 - 2025

A questionnaire or interview is a good way of getting **qualitative** feedback from your users. Qualitative feedback will be concerned with the opinions and views of your stakeholders. Some aspects of usability are quite difficult to quantify and might suit this particular method.

Below are three possible ways that the programmer could collect feedback about the usability of the menu screen. Can you see any possible issues with the use of any questions in terms of:

- **The quality of the feedback**
- **The ease of interpreting/analysing the feedback**

Question 1

What do you think of the menu screen?

Question 1

What do you think of the menu?

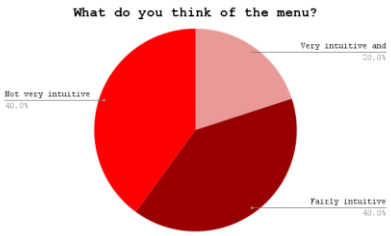
- ☐ Very intuitive and easy to follow
- ☐ Intuitive and easy to follow
- ☐ Fairly intuitive
- ☐ Not very intuitive or easy to follow
- ☐ Not at all intuitive

Question 1

What do you think of the menu?

Rate on scale of **1** (Not at all intuitive) to **10** (Very intuitive and easy to follow)

Your annotation should support the conclusions that you draw from your testing evidence. In the example below, a graph has been generated from a Google Form, and presented along with a written annotation.



From this feedback I can see that my stakeholders had differing views on the ease of use of my menu screen. 60% of my respondents said that it was either 'Fairly Intuitive' or 'Not Very Intuitive'. The minority of users responded by saying that it was intuitive – and they happened to be the most computer literate of my stakeholders. This suggests that I need to go back to my users to see what I can do to improve the menu screen.

As you reach the end of the testing process you need to ask yourself;

- **Are you able to successfully evaluate whether or not your project achieves what you set out to do?**
- **Do you have enough evidence to support your conclusions?**

If the answer to either question is no, you need to review your testing and ensure that you have carried it out in enough detail and in such a way that you are ready to review your success criteria.

© MrBComputingStuff 2022 - 2025

3.4.2 Success of the solution

"Perfection is achieved not when there is nothing more to add, but rather when there is nothing more to take away"

-Antoine de Saint-Exupery

So, you have almost reached the end of your project. If everything has gone the way that it should, you will now be ready to return to the success criteria that you that wrote what must seem like a lifetime ago!

It may be that parts of your project **haven't** gone well or that somethings don't work quite the way that they should. This doesn't mean that you have failed, though you should ensure that you address these areas in your evaluation. There is a bullet point tucked away in the mark scheme that is very important to consider in just this situation. You must make sure that you have…

"Provided comments on how any partially or unmet criteria could be addressed in further development."

3.4.2 a)
Use the test evidence from the development and post development process to evaluate the solution against the success criteria from the analysis

Now that the testing of your system is complete, you should be in a position where you can reflect on your outcomes and write about how successfully you have completed what you set out to do.

What you should do in this section:

- **EVALUATE** how successfully you achieved each of your success criteria
- **PROVIDE** or **CROSS REFERENCE** testing evidence that supports what you have written
- **EXPLAIN** what you could do to meet any unmet (or partially unmet) criteria, if you were to continue developing your work

In your analysis you came up with a set of smart targets you used to write a set of success criteria for your project.

Here are the two sample success criteria from the analysis part of this guide – we will look at how these could be evidenced in your evaluation.

SUCCESS CRITERIA	EVIDENCED BY	JUSTIFICATION
The player can change game controls	Screenshots/stakeholder witness statement	The ability to change the default control scheme has been highlighted as an important requirement by the users and is a key usability factor
Collision detection (between player / environment) works in all levels	Screen recording/stakeholder witness statement	Faulty collision detection would essentially mean a broken, unusable game. It has to work properly on each level

© MrBComputingStuff 2022 - 2025

Here is an evaluation of the second criterion in the table above:

SUCCESS CRITERIA	MET	PARTIALLY MET	NOT MET
Collision detection (between player / environment) works in all levels.	✔		
Functional/robustness testing evidence	Test #4 (screenshot and explanation of code) (link to video of game running)		
Post development testing evidence	Test #3 (Questionnaire response and statement from each stakeholder)		
REVIEW OF SUCCESS CRITERION I successfully completed this criterion, though it took until the second iteration of the program to do so (as evidenced by test #4) In the first iteration of the program, the player was still moving through walls and it wasn't until I revisited this part of the code in my second iteration of development, that I was able to rectify this mistake. I was pleased with the comments made by stakeholders in my post-development testing, since they made a point of saying how accurate the collision detection was.			
POSSIBLE FUTURE DEVELOPMENT Not necessary for this criterion			

Be clear about whether you have met the objective fully, partially or not at all

You do not need to generate NEW evidence or reproduce test evidence you have already used. It is much better to reference EXISTING tests

Use both types of testing to support your judgements

You should provide a written explanation to support your judgement

For any criteria that you DIDN'T meet (or that you only partially met) you should explain what you would do to address this, if you were to continue developing your project. You DO NOT HAVE TO MAKE THESE CHANGES – but your suggestions should be sensible and realistic as if you were going to implement them.

3.4.3 Describe the final product

"Usability is about people and how they understand and use things, not about technology"

-Steve Krug

As you approach the final few sections of the project, you are now considering the usability of your final solution, in terms of the GUI you created. There can be a distinct difference between how your UI works, and how your users **think** it will work. What is intuitive to you, the programmer, may not be intuitive to a user, especially if they are not confident when it comes to using computer applications.

You also need to evidence your final solution and consider maintainability of your program in the future, as well as how your project could be developed further.

3.4.3 a)
Provide annotated evidence of the usability features from the design, commenting on their effectiveness.

What you should do in this section:

- Provide evidence of the usability features that were implemented in the final version of your program
- Ensure that you evidenced **EACH** part of the system
- **JUSTIFY** how effectively you have implemented these features

Think back to section 3.2.2c – in the design section – where you thought about the usability of your program and considered how could create a GUI that would be suitable for your end user and what features you would need to include. In this section you are commenting about how well you implemented these features.

It should be straightforward to provide screenshots for each part of your system, and then write about how well you implemented it.

Your write-up should give clear reasons why you feel you implemented it successfully (or if appropriate, unsuccessfully). It would be even better if you could provide some stakeholder evidence, to support any statements you make.

According to the mark scheme you need to have…

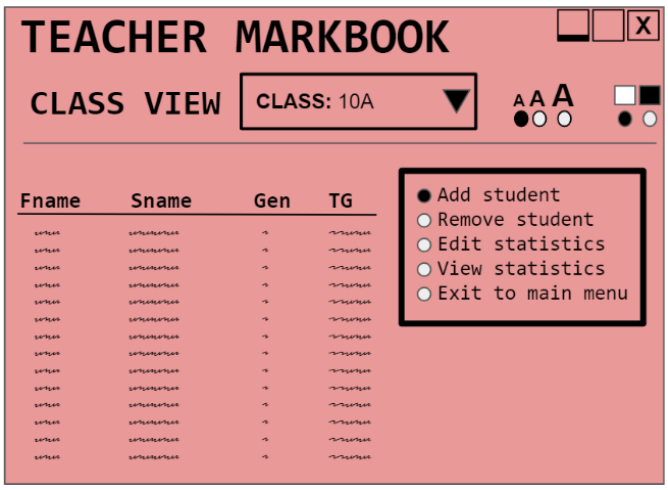
"Provided evidence of the usability features justifying their success, partial success or failure as effective usability features"

As with your success criteria, note that you need to determine the success, partial success or failure of the different usability aspects of your system.

Each separate screen (or in the case of a game, level) of your program needs to be evidenced.

© MrBComputingStuff 2022 - 2025

Below is an example from the teacher markbook program. Here, the programmer shows an example of one of the screens in the program (the class view), provides a commentary about how well this was implemented, and provides comments from the stakeholder to back up their views.



Here is the completed class view from the teacher markbook. I think this turned out well and matched the ideas in my design. **It at least partially meets the usability needs for my solution.** The drop-down menu at the top of the screen allows users to select a class without having to type anything in and removes the possibility of them mis-spelling a name or trying to type in a class. Addition the radio buttons worked well for different options worked well but my users suggested that a "CONFIRM" option would have been good, since the option is currently selected immediately. The users with accessibility needs did appreciate the options to change text size and to invert the colour scheme.

Stakeholder #1 comments	*"The user interface is very clear and well laid out. I was able to work in class view without any difficulty. The only issue I had was that the buttons on the right took me straight to the option and I sometimes clicked one in error. Could you possibly add a "confirm" button?*
Stakeholder #2 comments	*"I really appreciated the option to increase the text-size and invert the colour scheme. I found the screens so much easier to read with light colours on a dark background and set to the middle text size"*

© MrBComputingStuff 2022 - 2025

3.4.4 Maintenance and development

"Always code as if the person who ends up maintaining your code will be a violent psychopath who knows where you live"

-John F Woods

While there is some debate regarding the origin of the quote above, there can be no doubt regarding the quality of the advice it contains. Even though you have worked as a solo developer throughout this project, is extremely important that you demonstrate good development habits and that you are able to show that you understand the importance of maintainability.

When it comes to the future development of your program, it is worth reflecting on the "scope" of the original problem that you identified way back in the analysis. Hopefully your problem is one that you have been able to solve, while still having room for future development. This may be something that could be done to address any "limitations" in your work. This aspect of your evaluation forms the last part of your coursework.

3.4.4 a)
Discuss the maintainability of the solution.

First, let us remind ourselves what we mean by maintainability.

We will use the definition below, provided by the IEEE Standard Glossary of Software Engineering:

"The ease with which a software system or component can be modified to correct faults, improve performance or other attributes, or adapt to a changed environment."

What you should do in this section:

- Include the **completed code** for your project
- **Show evidence** of the use of appropriate maintainability techniques throughout the development process
- **Identify and discuss** any maintenance issues that might impact your ability to maintain the program.
- **Identify and discuss** any limitations that your project has

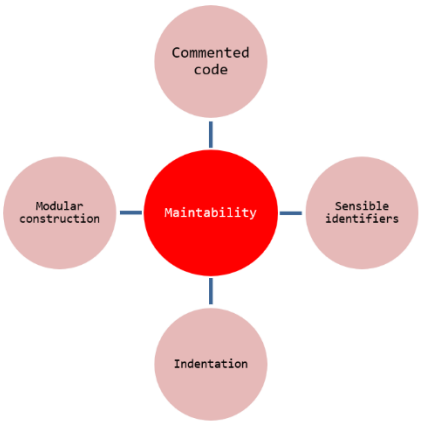
Maintenance falls into four categories:

CORRECTIVE	The need to fix any bugs or errors identified when the system is running
ADAPTIVE	The need to adapt the system in response to a change in the environment in which it is running. *(For example, a student markbook program will need to be changed if the school changes its grading system)*
PERFECTIVE	The ability to make future enhancements to a program, based on changing needs of the user *(For example, a teacher might request the ability to printout graphs of a student's performance, from within the markbook program)*
PREVENTATIVE	Measures take to pre-empt any errors or bugs occurring.

© MrBComputingStuff 2022 - 2025

You don't necessarily need to write a section for each type of maintenance – but it might be worth looking through your work a few times just to see if you think that your work is maintainable in all these ways.

There are a number of ways which can be used to ensure that your code is maintainable – many of them have already been referenced elsewhere in earlier sections of this project guidance.



COMMENTING has already been discussed in the development section and we looked at the principle of MODULAR program design and the use of SENSIBLE IDENTIFIERS in the design section.

Most IDE's automatically provide indentation, but if you are using one that does not, you need to ensure that you have made manual indents where appropriate (for example, when using selection, or iteration, when creating subprograms or classes)

You could give your code to another student and ask them to comment on its maintainability.

EVIDENCING MAINTAINABILITY TECHNIQUES

- ☐ A copy of your program code containing:
  - ☐ Appropriate comments
  - ☐ Sensible identifiers
  - ☐ Indentation
  - ☐ Evidence of modularity
- ☐ Information about any libraries that you have made use of (i.e. PyGame, GUIzero)
- ☐ Information about any graphics or sounds that you made use of, if they were created by a third party (for instance, for game making use of sprites, tile-sets and sound effects)

Don't make the examiner hunt for this evidence! Either annotate your listing or provide a separate written commentary

DISCUSSING MAINTAINABILITY ISSUES

There may be some issues that might cause maintainability issues going forward -

For example:

- ☐ Third-party libraries being changed or discontinued.
- ☐ Unrealistic stakeholder expectations
- ☐ The amount of time required to make significant changes.
- ☐ Availability of or access to hardware or software that might be needed.

Note – even if you do not feel that there any issues impacting maintainability of your program, you might want to explicitly say that and explain why, so that the examiner knows that you have considered it!

© MrBComputingStuff 2022 - 2025

DISCUSSING LIMITATIONS

Though the specification point for 3.4.4a doesn't make this clear, you also need to talk about limitations of your finished solution.

This is actually referenced in the mark scheme for the evaluation:

"Considered maintenance issues **and limitations** of the solution."

This is where you can go back to your analysis and your evaluation of the success criteria for ideas.

- Your **analysis (3.1.3c)** might give you ideas for areas to improve, based on aspects that you chose to leave out of your original problem. (This may have been because of time constraints or other reasons)
- Your **success criteria (3.4.2a)** might contain objectives that were either only partially met, or that you failed to meet at all.

Here is an example of how the programmer of the roguelike dungeon crawler could address some of the limitations of that project.

LIMITATIONS

1. Limited number of levels

In my analysis, I originally considered creating procedural levels for my game. It was time constraints and the additional algorithmic complexities of producing properly procedural (i.e. not just random) levels that caused me to opt for using pre-designed levels instead.

Even though I achieved this, and created three fully playable levels, there is very little longevity or replayability in the game when they are complete. The game is very limited in this aspect, certainly compared how a game with procedurally generated levels.

2. Monster AI is ineffective

This was a success criterion that I only partially met. While I did program a limited AI for my monsters, I based this around Dijkstra's algorithm which, in hindsight, was the wrong choice. Monsters seemed to respond slowly and did not see to follow the "optimum" routes to pursue the player.

Having looked at this again, I think I should have based my AI on A* algorithm. This would have resulted in a better response time in terms of monsters reacting to the player and then using the quickest route to find them.

Here the programmer refers back to the success criteria reviewed in 3.4.2.

© MrBComputingStuff 2022 - 2025

3.4.4 b)
Discuss potential further development of the solution.

What you should do in this section:

- **Suggest** ways in which your project could be developed further
- Ensure that your suggestions are **explained** in detail and that they show an appropriate line of reasoning

The final part of your project involves thinking about the scope for future development.

Even though you will not have to make any of the changes that you suggest, it is important that your suggestions are sensible and realistic.

Note that the final bullet point of mark-band 4 of the evaluation mark band, states that…

"There is a well-developed **line of reasoning** which is clear and logically structured. The information presented is **relevant** and **substantiated.**"

This means that your suggested improvements should make sense, given the context of the rest of your work.

Below is an example of a suggestion for how the roguelike dungeon crawler project could be developed further:

You may have had to leave a feature out due to time constraints. Is it something you could consider including in the future?

"If I had the opportunity to develop the project further, I would aim to create an algorithm that would produced procedurally generated levels. This is something I had to leave out of the project, due to the additional time it would have taken. I would have had to come up with a way of ensuring that the levels generated were suitably balanced and that they didn't contain elements that made them impossible to complete. My brief experimentation with procedural levels created a number of levels that could not be completed due to exits that led nowhere or a poor balance between monsters and power-ups/health drops.

My stakeholders had commented that my current game had little or no longevity once it was completed and procedurally generated levels was the something that they all suggested in the post-development feedback

In my analysis I looked at several existing roguelikes that contained this feature so I already know that this is something that would enhance my game significantly."

Referring back to ideas you might have discussed in your analysis, or feedback you had from stakeholders shows that your development ideas are based around sound reasoning.

© MrBComputingStuff 2022 - 2025

PROJECT CHECKLIST/SUGGESTED STRUCTURE

3.1.1 PROBLEM IDENTIFICATION

3.1.1a

☐ Description of your chosen problem
☐ Explanation of the key features that make it suitable for a computational solution
☐ Justification of the above

3.1.1b

☐ Explanation of the features that suit a computerised solution rather than a manual solution

3.1.2 STAKEHOLDERS

3.1.2a

☐ Identification of stakeholders
☐ Explanation of how the solution will meet their needs
☐ Identification of usability and accessibility features required

3.1.3 RESEARCH THE PROBLEM

3.1.3a

☐ Identification of at least THREE examples of solutions to problems that are similar to yours
☐ Identification of the key features of these solutions that could be included in your work
☐ Justification of reasons for including /not including features

3.13b

☐ Identification of essential features of your project
☐ Explanation of why these features are essential

3.1.3c

☐ Explanation of which features you have decided NOT to include in your project
☐ Explanation of the limitations of the features you HAVE included

3.1.4 SPECIFY THE PROPOSED SOLUTION

3.1.4a

☐ Identify/explain the requirements of your stakeholders
☐ Identify/explain the functions that your project must include
☐ Identify/explain hardware/software requirements of your project

3.1.4b

☐ List of measurable success criteria for your project
☐ Justification of each criterion

© MrBComputingStuff 2022 - 2025

3.2.1 DECOMPOSE THE PROBLEM

3.2.1a

☐ Decomposition diagram

3.2.2 DESCRIBE THE SOLUTION

3.2.2a

☐ Explanation of the structure of the solution (flowcharts/diagrams/ERD/DFD/IPSO diagrams)
☐ Justification of the structure of the solution

3.22b

☐ Algorithms for each part of the system
☐ Explanation of how each algorithm contributes towards the solution as a whole

3.2.2c

☐ GUI designs with annotation of how usability/accessibility have been considered

3.2.2d

☐ Identification of key variables/data structures/classes
☐ Justification of these choices

3.2.3 DESCRIBE THE APPROACH TO TESTING

3.2.3a

☐ Test plan (including a range of test types) for iterative testing
☐ Test plan (including user-testing scenarios) for post-development testing
☐ Justification for the inclusion of all the tests identified above

3.3.2 TESTING TO INFORM DEVELOPMENT

3.3.1a & b

☐ Evidence of iterative development – for EACH iteration/prototype include:
- Screenshots of code / program running
- Explanation of how code links back to your decomposition
- Testing and report on issues
- Stakeholder feedback
- Review of the iteration plus next steps

3.3.2a [included as testing evidence as part of 3.3.1a]

☐ Testing table (per iteration) including justification of test
☐ Annotated evidence of testing outcomes

3.3.2b [included with testing evidence]

☐ Annotated evidence of corrected code
☐ Annotated evidence of corrected code running
☐ Written justification of the corrective actions taken

© MrBComputingStuff 2022 - 2025

3.4.1 TESTING TO INFORM EVALUATION

3.4.1a

- ☐ Annotated evidence that the solution has been tested for FUNCTION
- ☐ Annotated evidence that the solution has been tested for ROBUSTNESS

3.4.1b

- ☐ Post development testing –
  - ○ Evidence that the solution has been tested for USABILITY

3.4.2 SUCCESS OF THE SOLUTION

3.4.2a

- ☐ Evaluate solution using success criteria from the analysis
- ☐ Cross-referencing to testing evidence to support evaluation
- ☐ Explanation of what could be done to address unmet/partially unmet criteria

3.4.3 DESCRIBE THE FINAL PRODUCT

3.4.3a

- ☐ Annotated evidence of usability features plus written comments on their effectiveness

3.4.4 MAINTENANCE AND DEVELOPMENT

3.4.4a

- ☐ Inclusion of completed code
- ☐ Evidence showing maintainability of code
- ☐ Identification/discussion of any maintenance issues
- ☐ Identification/discussion of any limitations of the solution

3.4.4b

- ☐ Suggestions regarding how the project could be developed further

© MrBComputingStuff 2022 - 2025